Sruthi Vellore (hv234)
Abhijeet Sahdev (as4673)
Subodh Dahal (sd983)

1 Goal

In Phase 3 of our project, we aim to finalize the development of the Morris Health Services (MHS) Application by implementing the database schema and application functionalities outlined in the project requirements. This phase involves the following goals:

- Finalise our EER diagram (1) for MHS along with its relational schema (2, documenting any changes.
- Develop menu-driven application for Employee and Facility Management, Patient Management, and Management Reporting, implementing specified functionalities with user-friendly interfaces for efficient navigation.
- Documenting our implementation and key challenges that we faced.

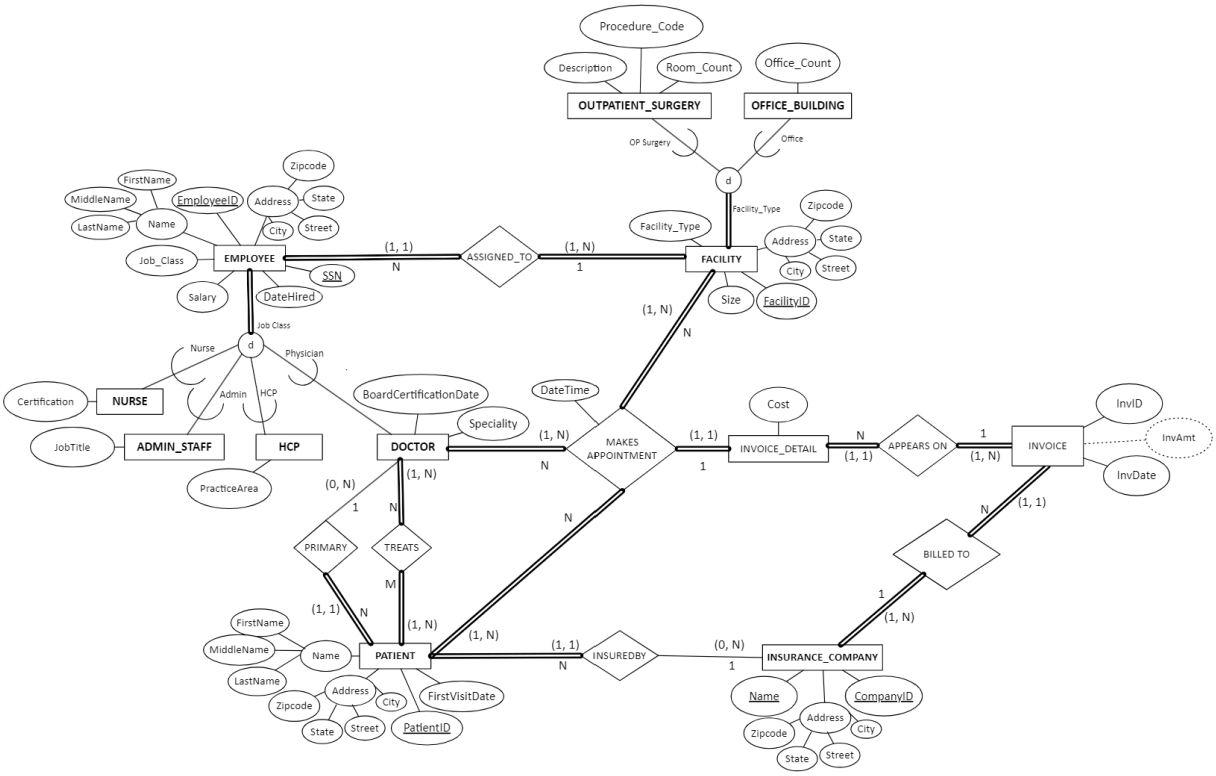


Figure 1: MHS ER diagram, no changes

2 added an attribute **Total_Cost** to **INVOICE** relation and **Reason** to **MAKES_APPOINTMENT**

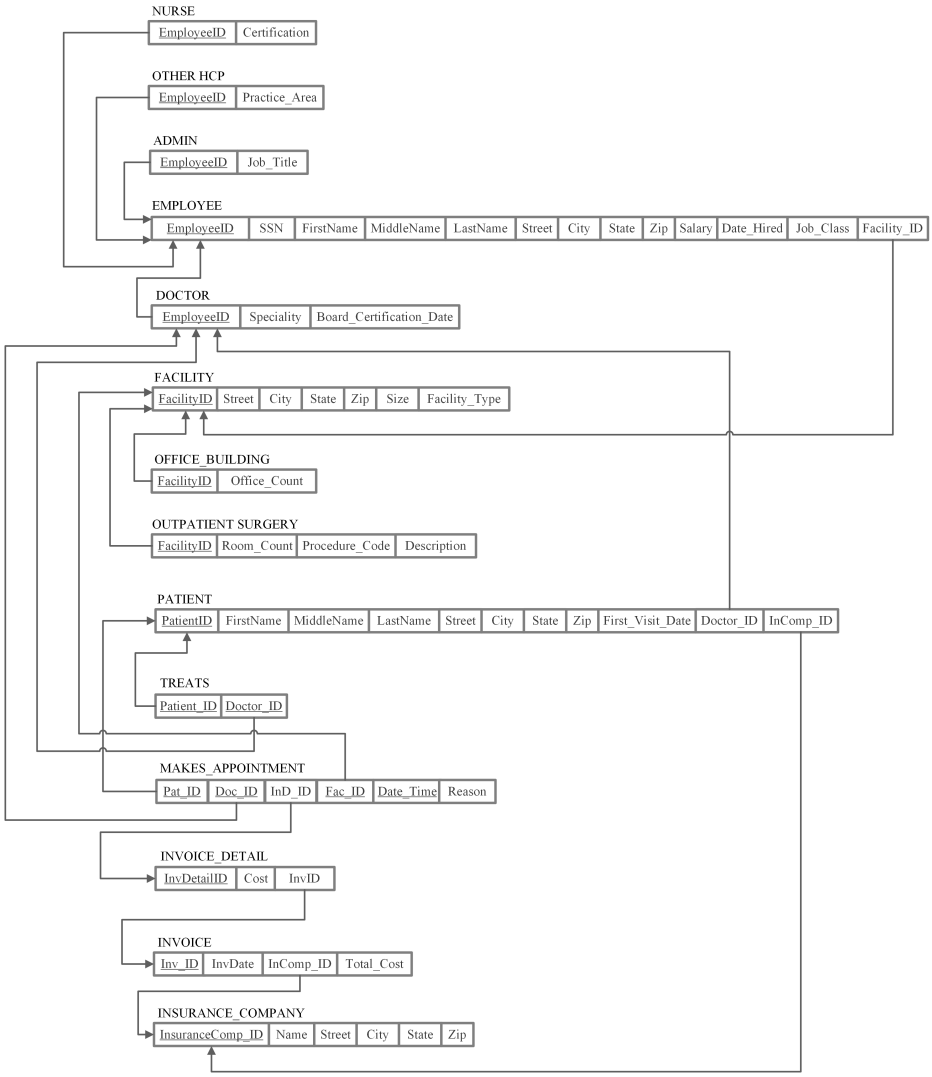


Figure 2: Relational Schema diagram

2 Description of Implementation and Challenges Faced

Our first step was to create the database in our local machines. We used MySQL along with an extension with Visual Studio Code called MySQL by Weijan Chen that helps us interface as clients. Then, we created our tables, specifying the domain constraints, key constraints, entity integrity constraints and referential integrity constraints. We iteratively added CHECK constraints and UNIQUE constraints to these as we programmed our application. With each iteration, we reflected its update on our database through the front-end iterations. The implementation of the Morris Health Services project using Python Django involved the following key components:

- The Connection to the database involves two steps, setting.py (hidden on our repository on Github) file with the Database configuration parameters as a ”DATABASE” dictionary and we have written a Database utility file ”db_utils.py” to connect and execute SQL queries. ”execute_query” function defined in ”db_utils.py” executs queries, handles exceptions and ensure proper error logging.

- ”View.py” file contains the business logic of the application. Each function is identified by its specific URL route defined in ’urlpatterns’ list. These views interact with the database through the ’db_utils’ module.
- URL routing is configured in ’urlpatterns’ list of the ’urls.py’ module. Each URL pattern is mapped to its corresponding view function to navigate within the application.
- The frontend of the application is implemented using HTML. The ’templates’ folder contains HTML files used for rendering user interface. The subfolders ’employee’, ’facility’, ’insurance’, ’patient’ and ’reports’, includes pages for employee management, facility management, patient management and reporting.

Initially, we didn’t close our connections to the the databse if there was an error in executing a query, which left multiple connections open for a long time. So, when trying to update an appointment, our MySQL database could not handle the operation to find the composite key across too many entries. We handled that by first, killing all the threads (connections) with our database and then, implemented the standard query execution sequence, open, committing and closing, irrespective of the result of the query, using try, catch and finally blocks. Our main challenge was transaction management, when creating daily invoices after an appointment was complemented or updating employees/facilities and reflecting changes in the subclasses.

2 Create Table Statements

```
DROP TABLE IF EXISTS MAKES_APPOINTMENT;
DROP TABLE IF EXISTS TREATS;
DROP TABLE IF EXISTS INVOICE_DETAIL;
DROP TABLE IF EXISTS OUTPATIENT_SURGERY;
DROP TABLE IF EXISTS OFFICE_BUILDING;
DROP TABLE IF EXISTS INVOICE;
DROP TABLE IF EXISTS PATIENT;
DROP TABLE IF EXISTS DOCTOR;
DROP TABLE IF EXISTS NURSE;
DROP TABLE IF EXISTS OTHER_HCP;
DROP TABLE IF EXISTS ADMIN_STAFF;
DROP TABLE IF EXISTS EMPLOYEE;
DROP TABLE IF EXISTS FACILITY;
DROP TABLE IF EXISTS INSURANCE_COMPANY;

CREATE TABLE FACILITY (
    Facility_ID INT AUTO_INCREMENT PRIMARY KEY,
    Street VARCHAR(255) NOT NULL,
    City VARCHAR(255) NOT NULL,
    State VARCHAR(2) NOT NULL,
    Zip VARCHAR(5) NOT NULL,
    MaxSize INT,
    Facility_Type VARCHAR(50) NOT NULL,
    CHECK (Facility_Type IN ('OP Surgery', 'Office')),
    CHECK (Zip REGEXP '^[0-9]{5}$')
);
```

```sql
CREATE TABLE INSURANCE_COMPANY (
    InsuranceComp_ID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(255) NOT NULL UNIQUE,
    Street VARCHAR(255) NOT NULL,
    City VARCHAR(255) NOT NULL,
    State VARCHAR(2) NOT NULL,
    Zip VARCHAR(5) NOT NULL
);

CREATE TABLE EMPLOYEE (
    EmployeeID INT AUTO_INCREMENT PRIMARY KEY,
    SSN CHAR(9) UNIQUE NOT NULL,
    FirstName VARCHAR(255) NOT NULL,
    MiddleName VARCHAR(255),
    LastName VARCHAR(255) NOT NULL,
    Street VARCHAR(255) NOT NULL,
    City VARCHAR(255) NOT NULL,
    State VARCHAR(255) NOT NULL,
    Zip VARCHAR(5) NOT NULL,
    Salary DECIMAL(10, 2) CHECK (Salary >= 0),
    Date_Hired DATE NOT NULL,
    Job_Class VARCHAR(50) NOT NULL,
    Fac_ID INT NOT NULL,
    FOREIGN KEY (Fac_ID) REFERENCES FACILITY(Facility_ID),
    CHECK (SSN REGEXP '^[0-9]{9}$'),
    CHECK (Job_Class IN ('Doctor', 'Nurse', 'HCP', 'Admin')),
    CHECK (Zip REGEXP '^[0-9]{5}$')
);

CREATE TABLE DOCTOR (
    EmployeeID INT PRIMARY KEY,
    Speciality VARCHAR(255) NOT NULL,
    Board_Certification_Date DATE NOT NULL,
    FOREIGN KEY (EmployeeID) REFERENCES EMPLOYEE(EmployeeID)
    ON DELETE CASCADE
);

CREATE TABLE NURSE (
    EmployeeID INT PRIMARY KEY,
    Certification VARCHAR(255) NOT NULL,
    FOREIGN KEY (EmployeeID) REFERENCES EMPLOYEE(EmployeeID)
    ON DELETE CASCADE
);

CREATE TABLE OTHER_HCP (
    EmployeeID INT PRIMARY KEY,
    Practice_Area VARCHAR(255) NOT NULL,
```

```
    FOREIGN KEY (EmployeeID) REFERENCES EMPLOYEE(EmployeeID)
    ON DELETE CASCADE
);

CREATE TABLE ADMIN_STAFF (
    EmployeeID INT PRIMARY KEY,
    Job_Title VARCHAR(255) NOT NULL,
    FOREIGN KEY (EmployeeID) REFERENCES EMPLOYEE(EmployeeID)
    ON DELETE CASCADE
);

CREATE TABLE PATIENT (
    Patient_ID INT AUTO_INCREMENT PRIMARY KEY,
    FirstName VARCHAR(255) NOT NULL,
    MiddleName VARCHAR(255),
    LastName VARCHAR(255) NOT NULL,
    Street VARCHAR(255) NOT NULL,
    City VARCHAR(255) NOT NULL,
    State VARCHAR(2) NOT NULL,
    Zip VARCHAR(5) NOT NULL,
    First_Visit_Date DATE NOT NULL,
    Doctor_ID INT NOT NULL,
    InComp_ID INT NOT NULL,
    FOREIGN KEY (Doctor_ID) REFERENCES DOCTOR(EmployeeID),
    FOREIGN KEY (InComp_ID) REFERENCES
       INSURANCE_COMPANY(InsuranceComp_ID),
    CHECK (Zip REGEXP '^[0-9]{5}$')
);

CREATE TABLE OFFICE_BUILDING (
    Facility_ID INT PRIMARY KEY,
    Office_Count INT CHECK (Office_Count >= 0),
    FOREIGN KEY (Facility_ID) REFERENCES FACILITY(Facility_ID)
    ON DELETE CASCADE
);

CREATE TABLE OUTPATIENT_SURGERY (
    Facility_ID INT PRIMARY KEY,
    Room_Count INT CHECK (Room_Count >= 0),
    Procedure_Code VARCHAR(255) NOT NULL,
    Description VARCHAR(1000),
    FOREIGN KEY (Facility_ID) REFERENCES FACILITY(Facility_ID)
    ON DELETE CASCADE
);

CREATE TABLE MAKES_APPOINTMENT (
    Pat_ID INT,
```

```
    Doc_ID INT,
    Fac_ID INT,
    Date_Time TIMESTAMP,
    InD_ID INT NOT NULL,
    Reason VARCHAR(1000) NOT NULL,
    PRIMARY KEY (Pat_ID, Doc_ID, Fac_ID, Date_Time),
    FOREIGN KEY (Pat_ID) REFERENCES PATIENT(Patient_ID),
    FOREIGN KEY (Doc_ID) REFERENCES DOCTOR(EmployeeID),
    FOREIGN KEY (Fac_ID) REFERENCES FACILITY(Facility_ID),
    FOREIGN KEY (InD_ID) REFERENCES INVOICE_DETAIL(Inv_ID)
);

ALTER TABLE MAKES_APPOINTMENT
ADD CONSTRAINT Doctor_Time UNIQUE (Doc_ID, Date_Time);

ALTER TABLE MAKES_APPOINTMENT
ADD CONSTRAINT Patient_Time UNIQUE (Pat_ID, Date_Time);

CREATE TABLE TREATS (
    Patient_ID INT,
    Doctor_ID INT,
    PRIMARY KEY (Patient_ID, Doctor_ID),
    FOREIGN KEY (Patient_ID) REFERENCES PATIENT(Patient_ID),
    FOREIGN KEY (Doctor_ID) REFERENCES DOCTOR(EmployeeID)
);

CREATE TABLE INVOICE_DETAIL (
    InvDetailID INT AUTO_INCREMENT PRIMARY KEY,
    Cost DECIMAL(10, 2) CHECK (Cost >= 0),
    Inv_ID INT NOT NULL,
    FOREIGN KEY (Inv_ID) REFERENCES INVOICE(Inv_ID)
    ON DELETE CASCADE
);

CREATE TABLE INVOICE (
    Inv_ID INT AUTO_INCREMENT PRIMARY KEY,
    InvDate DATE,
    InComp_ID INT NOT NULL,
    FOREIGN KEY (InComp_ID) REFERENCES
       INSURANCE_COMPANY(InsuranceComp_ID),
    Total_Cost DECIMAL(10, 2) DEFAULT(0)
    CHECK (Total_Cost >= 0)
);

DROP TRIGGER IF EXISTS UpdateDaiyInvoice;

DELIMITER $$
```

```
CREATE TRIGGER UpdateDaiyInvoice
AFTER INSERT ON INVOICE_DETAIL
FOR EACH ROW
BEGIN
    UPDATE INVOICE
    SET Total_Cost = Total_Cost + NEW.Cost
    WHERE Inv_ID = NEW.Inv_ID;

    INSERT INTO TriggerLog (TableName, ActionTaken)
    VALUES ('INVOICE_DETAIL', 'Updated Total_Cost in INVOICE');
END$$
DELIMITER ;
```

3 INSERT Statements

```
    INSERT INTO FACILITY (Street, City, State, Zip, MaxSize,
       Facility_Type) VALUES
('123 Main St', 'Springfield', 'IL', '62701', 150, 'Office'),
('124 Main St', 'Springfield', 'IL', '62702', 100, 'Office'),
('125 Main St', 'Springfield', 'IL', '62703', 200, 'Office'),
('126 Main St', 'Springfield', 'IL', '62704', 120, 'Office'),
('127 Main St', 'Springfield', 'IL', '62705', 130, 'Office'),
('128 Main St', 'Springfield', 'IL', '62706', 140, 'Office'),
('129 Main St', 'Springfield', 'IL', '62707', 110, 'Office'),
('130 Main St', 'Springfield', 'IL', '62708', 160, 'Office'),
('131 Main St', 'Springfield', 'IL', '62709', 170, 'Office'),
('132 Main St', 'Springfield', 'IL', '62710', 180, 'Office'),
('133 Main St', 'Springfield', 'IL', '62711', 190, 'Office'),
('134 Pine St', 'Lakeview', 'TX', '75001', 80, 'OP Surgery'),
('135 Pine St', 'Lakeview', 'TX', '75002', 90, 'OP Surgery'),
('136 Pine St', 'Lakeview', 'TX', '75003', 75, 'OP Surgery'),
('137 Pine St', 'Lakeview', 'TX', '75004', 65, 'OP Surgery'),
('138 Pine St', 'Lakeview', 'TX', '75005', 85, 'OP Surgery'),
('139 Pine St', 'Lakeview', 'TX', '75006', 95, 'OP Surgery'),
('140 Pine St', 'Lakeview', 'TX', '75007', 70, 'OP Surgery'),
('141 Pine St', 'Lakeview', 'TX', '75008', 60, 'OP Surgery');

INSERT INTO OFFICE_BUILDING (Facility_ID, Office_Count) VALUES
(1, 10),
(2, 9),
(3, 8),
(4, 7),
(5, 6),
(6, 5),
(7, 4),
(8, 3),
(9, 2),
```

```
(10, 1),
(11, 3);

INSERT INTO OUTPATIENT_SURGERY (Facility_ID, Room_Count,
   Procedure_Code, Description) VALUES
(12, 10, '12345', 'Routine Appendectomy'),
(13, 9, '23456', 'Cataract Surgery'),
(14, 8, '34567', 'Knee Arthroscopy'),
(15, 7, '45678', 'Laser Eye Surgery'),
(16, 6, '56789', 'Gallbladder Removal'),
(17, 5, '67890', 'Hernia Repair'),
(18, 4, '78901', 'Tonsillectomy'),
(19, 3, '89012', 'Colonoscopy');

INSERT INTO INSURANCE_COMPANY (Name, Street, City, State, Zip) VALUES
('GoodHealth Ins', '100 Health Blvd', 'Capital City', 'DC', '20001'),
('SecureLife', '200 Wellness Dr', 'Oldtown', 'OK', '73034'),
('FamilyCare', '300 Safe St', 'Trustville', 'CA', '94016'),
('BudgetShield', '400 Cover Rd', 'Savecity', 'NY', '10001');

INSERT INTO EMPLOYEE (SSN, FirstName, MiddleName, LastName, Street,
   City, State, Zip, Salary, Date_Hired, Job_Class, Fac_ID) VALUES
('123456789', 'John', 'D', 'Doe', '123 Work St', 'Worktown', 'CA',
   '94016', 120000.00, '2020-01-10', 'Doctor', 1),
('987654321', 'Jane', 'E', 'Smith', '456 Job Ave', 'Jobville', 'NY',
   '10001', 95000.00, '2021-02-15', 'Nurse', 2),
('234567890', 'Emily', 'F', 'Jones', '789 Career Blvd', 'Workcity',
   'TX', '75001', 50000.00, '2022-03-10', 'Admin', 3),
('345678901', 'Michael', 'G', 'Brown', '101 Admin Rd', 'Adminville',
   'FL', '32250', 60000.00, '2020-04-20', 'HCP', 1),
('456789012', 'Chloe', 'H', 'Davis', '202 Staff St', 'Stafftown',
   'CA', '94016', 120000.00, '2020-05-15', 'Doctor', 4),
('567890123', 'Luke', 'I', 'Wilson', '303 Job Way', 'Jobburgh', 'NY',
   '10001', 95000.00, '2021-06-18', 'Nurse', 5),
('678901234', 'Olivia', 'J', 'Martinez', '404 Career Ct', 'Workshire',
   'TX', '75001', 50000.00, '2022-07-22', 'Admin', 2),
('789012345', 'James', 'K', 'Taylor', '505 Admin Ave', 'Adminburg',
   'FL', '32250', 60000.00, '2020-08-25', 'HCP', 3),
('890123456', 'Isabella', 'L', 'Thomas', '606 Staff Rd', 'Staffville',
   'CA', '94016', 120000.00, '2020-09-28', 'Doctor', 6),
('901234567', 'Ethan', 'M', 'Harris', '707 Job Ln', 'Jobland', 'NY',
   '10001', 95000.00, '2021-10-30', 'Nurse', 7),
('012345678', 'Sophia', 'N', 'Moore', '808 Career Way', 'Worktown',
   'TX', '75001', 50000.00, '2022-11-01', 'Admin', 4),
('123456780', 'Jacob', 'O', 'Jackson', '909 Admin St', 'Admincity',
   'FL', '32250', 60000.00, '2020-12-03', 'HCP', 5),
('234567891', 'Mia', 'P', 'Lee', '121 Staff Ave', 'Staffshire', 'CA',
```

```
   '94016', 120000.00, '2021-01-04', 'Doctor', 8),
('345678902', 'William', 'Q', 'Anderson', '232 Job Rd', 'Jobville',
   'NY', '10001', 95000.00, '2022-02-05', 'Nurse', 9),
('456789013', 'Ava', 'R', 'Thomas', '343 Career Blvd', 'Workcity',
   'TX', '75001', 50000.00, '2023-03-07', 'Admin', 6),
('567890124', 'Noah', 'S', 'Miller', '454 Admin Way', 'Adminburgh',
   'FL', '32250', 60000.00, '2024-04-08', 'HCP', 7),
('678901235', 'Lily', 'T', 'Davis', '565 Staff Ct', 'Stafftown', 'CA',
   '94016', 120000.00, '2025-05-10', 'Doctor', 10),
('789012346', 'Benjamin', 'U', 'Wilson', '676 Job Ave', 'Jobshire',
   'NY', '10001', 95000.00, '2026-06-11', 'Nurse', 11),
('890123457', 'Emma', 'V', 'Martinez', '787 Career Rd', 'Workville',
   'TX', '75001', 50000.00, '2027-07-13', 'Admin', 8),
('901234568', 'Jack', 'W', 'Taylor', '898 Admin Ln', 'Adminland',
   'FL', '32250', 60000.00, '2028-08-14', 'HCP', 9);

INSERT INTO DOCTOR (EmployeeID, Speciality, Board_Certification_Date)
   VALUES
(1, 'Cardiology', '2020-01-10'),
(5, 'Dermatology', '2020-05-15'),
(9, 'Endocrinology', '2020-09-28'),
(13, 'Gastroenterology', '2021-01-04'),
(17, 'Hematology', '2021-05-10');

INSERT INTO NURSE (EmployeeID, Certification) VALUES
(2, 'RN'),
(6, 'LPN'),
(10, 'CNA'),
(14, 'RN'),
(18, 'LPN');

INSERT INTO ADMIN_STAFF (EmployeeID, Job_Title) VALUES
(3, 'Receptionist'),
(7, 'Secretary'),
(11, 'Clerk'),
(15, 'Receptionist'),
(19, 'Secretary');

INSERT INTO OTHER_HCP (EmployeeID, Practice_Area) VALUES
(4, 'Physical Therapy'),
(8, 'Occupational Therapy'),
(12, 'Speech Therapy'),
(16, 'Physical Therapy'),
(20, 'Occupational Therapy');

INSERT INTO PATIENT (FirstName, MiddleName, LastName, Street, City,
   State, Zip, First_Visit_Date, Doctor_ID, InComp_ID) VALUES
```

```
('Alice', 'F', 'Johnson', '789 Patient Rd', 'Healtown', 'FL', '32250',
   '2023-01-10', 1, 1),
('Bob', 'G', 'Lee', '101 Recovery Ln', 'Medcity', 'TX', '75070',
   '2023-02-20', 5, 1),
('Charlie', 'H', 'Clark', '234 Health St', 'Wellville', 'CA', '94016',
   '2023-03-15', 9, 2),
('Diana', 'I', 'Wright', '345 Care Ave', 'Curetown', 'NY', '10001',
   '2023-04-10', 13, 2),
('Evan', 'J', 'Morris', '456 Wellness Blvd', 'Aidcity', 'VA', '24502',
   '2023-05-05', 17, 3),
('Fiona', 'K', 'Smith', '567 Help Rd', 'Nursetown', 'NC', '27514',
   '2023-06-21', 1, 3),
('George', 'L', 'Baker', '678 Assist St', 'Recoveryville', 'GA',
   '30301', '2023-07-15', 5, 4),
('Hannah', 'M', 'Gonzalez', '789 Heal Ln', 'Therapytown', 'MA',
   '02101', '2023-08-10', 9, 4),
('Ian', 'N', 'Davis', '890 Save Ave', 'Doccity', 'AZ', '85001',
   '2023-09-20', 13, 1),
('Julia', 'O', 'Martinez', '901 Medicine Rd', 'Clinictown', 'IL',
   '60007', '2023-10-15', 17, 2);

INSERT INTO TREATS (Patient_ID, Doctor_ID) VALUES
(1, 1),
(2, 5),
(3, 9),
(4, 13),
(5, 17),
(6, 1),
(7, 5),
(8, 9),
(9, 13),
(10, 17);


INSERT INTO INVOICE VALUES
(1,'2024-04-28',1, 0),
(2,'2024-04-29',1, 0),
(3,'2024-04-30',1, 0),
(4,'2024-05-01',1, 0),
(5,'2024-05-02',2, 0),
(6,'2024-04-29',2, 0),
(7,'2024-05-01',3, 0),
(8,'2024-04-29',3, 0),
(9,'2024-04-30',2, 0);

INSERT INTO INVOICE_DETAIL VALUES
(1,432.00,1),
```

```
(2,421.00,1),
(3,543.00,2),
(4,521.00,3),
(5,345.00,4),
(6,234.00,1),
(7,531.00,4),
(8,453.00,5),
(9,432.00,6),
(10,42.00,7),
(11,342.00,8),
(12,789.00,2),
(13,468.00,9),
(14,759.00,9),
(15,1000.00,5),
(16,1000.00,3),
(17,1000.00,5);

INSERT INTO MAKES_APPOINTMENT VALUES
(2,5,2,'2024-04-28 04:00:00',1,'Fever'),
(1,1,1,'2024-04-29 03:33:00',2,'Headache'),
(1,9,3,'2024-04-29 04:00:00',3,'Cough'),
(1,1,1,'2024-04-30 07:45:00',4,'Back Pain'),
(1,1,1,'2024-05-01 09:55:00',5,'Stomachache'),
(1,9,1,'2024-04-28 09:55:00',6,'Sore Throat'),
(1,17,11,'2024-05-01 04:00:00',7,'Fatigue'),
(3,9,15,'2024-05-03 03:33:00',8,'Allergies'),
(4,5,7,'2024-04-29 15:11:00',9,'Sprained Ankle'),
(5,5,5,'2024-05-01 08:00:00',10,'Common Cold'),
(6,13,17,'2024-04-29 10:44:00',11,'Migraine'),
(9,9,8,'2024-04-29 07:44:00',12,'Arthritis'),
(10,5,19,'2024-04-30 12:40:00',13,'Diabetes'),
(10,9,8,'2024-05-01 03:00:00',14,'High Blood Pressure'),
(10,13,4,'2024-05-02 04:00:00',15,'Asthma'),
(1,13,5,'2024-04-30 04:55:00',16,'Acid Reflux'),
(3,9,9,'2024-05-03 03:00:00',17,'Anxiety');
```

5 Code Source

5.1 settings.py

```
# default settings of Django App
DATABASES = {
    'default': {
       'ENGINE': 'django.db.backends.mysql',
        'NAME': 'MHS', # Change this to your MySQL database name
        'USER': 'root',
        'PASSWORD': '', # Change this to your MySQL password
        'HOST': 'localhost',
```

```
        'PORT': '3306',
    }
}
```

5.2 db_utils.py

```
import MySQLdb
import logging
from django.conf import settings

# Configure logging
logging.basicConfig(level=logging.DEBUG)
logger = logging.getLogger(__name__)

def execute_query(query, params=None, fetchone=False, fetchall=False,
   insert_new=False):
    db = MySQLdb.connect(
        host=settings.DATABASES['default']['HOST'],
        user=settings.DATABASES['default']['USER'],
        passwd=settings.DATABASES['default']['PASSWORD'],
        db=settings.DATABASES['default']['NAME']
    )
    cursor = db.cursor(MySQLdb.cursors.DictCursor)
    try:
        logger.debug("Executing query: %s", query)
        logger.debug("Executing params: %s", params)
        cursor.execute(query, params or ())

        if fetchone:
            result = cursor.fetchone()
        elif fetchall:
            result = cursor.fetchall()
        elif insert_new:
            result = cursor.lastrowid
        else:
            result = "Success"

        db.commit()
        logger.debug("Query result: %s", result)
        return result
    except MySQLdb.Error as e:
        db.rollback()
        logger.error("Error in executing query: %s", e)
        return "Error"
    finally:
        cursor.close()
        db.close()
```

5.3 urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
    path('reports/1', views.revenue_report_by_facility,
       name='view_report1'),
    path('reports/2', views.appointments_by_date_and_physician,
       name='view_report2'),
    path('reports/3', views.appointments_by_time_period_and_facility,
       name='view_report3'),
    path('reports/4', views.best_revenue_days_for_month,
       name='view_report4'),
    path('reports/5',
       views.average_daily_revenue_by_insurance_company,
       name='view_report5'),
    #Facility Views
    path('facility/', views.view_facility, name='view_facility'),
    path('facility/ob', views.view_ob, name='view_ob'),
    path('facility/ops', views.view_ops, name='view_ops'),
    path('facility/add', views.create_facility,
       name='create_facility'),
    path('facility/edit', views.edit_facility, name='edit_facility'),
    #Employee Views
    path('emp/doctor/', views.view_doctor, name='view_doctor'),
    path('emp/nurse/', views.view_nurse, name='view_nurse'),
    path('emp/admin_staff/', views.view_admin_staff,
       name='view_admin_staff'),
    path('emp/hcp/', views.view_hcp, name='view_hcp'),
    path('emp/add_employee/', views.add_employee, name='add_employee'),
    path('emp/edit_employee/', views.edit_employee,
       name='edit_employee'),
    path('emp/', views.view_employee, name='view_employee'),
    #Patient Views
    path('patient/add_patient/', views.add_patient,
       name='add_patient'),
    path('patient/', views.view_patient, name='view_patient'),
    path('patient/edit_patient/', views.edit_patient,
       name='edit_patient'),
    path('patient/make_appointment', views.make_appointment,
       name='make_appointment'),
    path('patient/update_appointment', views.update_appointment,
       name='update_appointment'),
    path('patient/daily_inv', views.daily_inv, name='daily_inv'),
    path('patient/all_appointment/', views.view_appointment,
```

```
        name='view_appointment'),
    #Insurance Views
    path('insurance/', views.view_insurance, name='view_insurance'),
    path('insurance/add_insurance', views.add_insurance,
        name='add_insurance'),
    path('insurance/edit_insurance', views.edit_insurance,
        name='edit_insurance'),
]
```

5.4 views.py

```
from django.shortcuts import redirect, render

from .manageFacility import delete_old_facility_details,
    get_current_facility_type, insert_office_details,
    insert_ops_details, update_office_details, update_ops_details

from .manageEmployee import insert_admin_staff_details,
    insert_doctor_details, insert_nurse_details,
    insert_other_hcp_details, update_employee_details
from .db_utils import execute_query
from django.core.paginator import Paginator
from django.http import HttpResponse, JsonResponse
import datetime
from django.contrib import messages

def index(request):
    return render(request, 'intro.html')

# Employee Management Views
def view_employee(request):
    sql = "SELECT * FROM EMPLOYEE"
    employees = execute_query(sql, fetchall=True)

    paginator = Paginator(employees, 5)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)

    return render(request, 'employee/employee.html', {'employees':
        page_obj})

def view_doctor(request):
    base_sql = """
        SELECT
            D.*,
            E.FirstName AS EmpFirstName,
            E.MiddleName AS EmpMiddleName,
```

```
            E.LastName AS EmpLastName,
            E.Street AS EmpStreet,
            E.City AS EmpCity,
            E.State AS EmpState,
            E.Zip AS EmpZip,
            E.Salary AS EmpSalary,
            E.Date_Hired AS EmpDateHired,
            E.SSN AS SSN,
            E.Fac_ID AS EmpFacilityID
        FROM
            DOCTOR D
        INNER JOIN
            EMPLOYEE E ON D.EmployeeID = E.EmployeeID
    """

    doctors = execute_query(base_sql, fetchall=True)

    paginator = Paginator(doctors, 5)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)

    return render(request, 'employee/doctor.html', {'doctors':
       page_obj})

def view_nurse(request):
    base_sql = """
        SELECT
            N.*,
            E.FirstName AS EmpFirstName,
            E.MiddleName AS EmpMiddleName,
            E.LastName AS EmpLastName,
            E.Street AS EmpStreet,
            E.City AS EmpCity,
            E.State AS EmpState,
            E.Zip AS EmpZip,
            E.Salary AS EmpSalary,
            E.Date_Hired AS EmpDateHired,
            E.SSN AS SSN,
            E.Fac_ID AS EmpFacilityID
        FROM
            NURSE N
        INNER JOIN
            EMPLOYEE E ON N.EmployeeID = E.EmployeeID
    """

    nurses = execute_query(base_sql, fetchall=True)
```

```
    paginator = Paginator(nurses, 5)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)

    return render(request, 'employee/nurse.html', {'nurses': page_obj})

def view_admin_staff(request):
    base_sql = """
        SELECT
            A.*,
            E.FirstName AS EmpFirstName,
            E.MiddleName AS EmpMiddleName,
            E.LastName AS EmpLastName,
            E.Street AS EmpStreet,
            E.City AS EmpCity,
            E.State AS EmpState,
            E.Zip AS EmpZip,
            E.Salary AS EmpSalary,
            E.Date_Hired AS EmpDateHired,
            E.SSN AS SSN,
            E.Fac_ID AS EmpFacilityID
        FROM
            ADMIN_STAFF A
        INNER JOIN
            EMPLOYEE E ON A.EmployeeID = E.EmployeeID
    """
    admin_staff = execute_query(base_sql, fetchall=True)

    paginator = Paginator(admin_staff, 5)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)

    return render(request, 'employee/admin_staff.html',
       {'admin_staff': page_obj})

def view_hcp(request):
    base_sql = """
        SELECT
            H.*,
            E.FirstName AS EmpFirstName,
            E.MiddleName AS EmpMiddleName,
            E.LastName AS EmpLastName,
            E.Street AS EmpStreet,
            E.City AS EmpCity,
            E.State AS EmpState,
            E.Zip AS EmpZip,
            E.Salary AS EmpSalary,
```

```
            E.Date_Hired AS EmpDateHired,
            E.SSN AS SSN,
            E.Fac_ID AS EmpFacilityID
        FROM
            OTHER_HCP H
        INNER JOIN
            EMPLOYEE E ON H.EmployeeID = E.EmployeeID
    """
    hcps = execute_query(base_sql, fetchall=True)

    paginator = Paginator(hcps, 5)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)

    return render(request, 'employee/hcp.html', {'other_hcp':
       page_obj})

def add_employee(request):
    if request.method == 'POST':
        ssn = request.POST.get('ssn')
        first_name = request.POST.get('first_name')
        middle_name = request.POST.get('middle_name', '')
        last_name = request.POST.get('last_name')
        street = request.POST.get('street')
        city = request.POST.get('city')
        state = request.POST.get('state')
        zip_code = request.POST.get('zip')
        salary = request.POST.get('salary')
        date_hired = request.POST.get('date_hired')
        job_class = request.POST.get('job_class')
        facility_id = request.POST.get('facility_id')

        # Inserting into EMPLOYEE table
        employee_sql = """
        INSERT INTO EMPLOYEE (SSN, FirstName, MiddleName, LastName,
           Street, City, State, Zip, Salary, Date_Hired, Job_Class,
           Fac_ID)
        VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
        """
        employee_params = (ssn, first_name, middle_name, last_name,
           street, city, state, zip_code, salary, date_hired,
           job_class, facility_id)
        result = execute_query(employee_sql, params=employee_params,
           insert_new=True)
        print(result)
        if result == "Error":
            messages.error(request, 'Error inserting employee. Please
```

```
                try again.')
        else:
            employee_id = result
            # Inserting into specific table based on job class
            kwargs = {}
            if job_class == 'Doctor':
                kwargs['speciality'] = request.POST.get('speciality')
                kwargs['board_certification_date'] =
                    request.POST.get('board_certification_date')
                result = insert_doctor_details(employee_id, **kwargs)
            elif job_class == 'Nurse':
                kwargs['certification'] =
                    request.POST.get('certification')
                result = insert_nurse_details(employee_id, **kwargs)
            elif job_class == 'HCP':
                kwargs['practice_area'] =
                    request.POST.get('practice_area')
                result = insert_other_hcp_details(employee_id,
                    **kwargs)
            elif job_class == 'Admin':
                kwargs['job_title'] = request.POST.get('job_title')
                result = insert_admin_staff_details(employee_id,
                    **kwargs)

            if  result != "Error" :
                messages.success(request, 'Employee details inserted
                    successfully.')
            else:
                messages.error(request, 'Error inserting employee
                    proffesion. Please try again.')
        return redirect('add_employee')

    # Fetching facility IDs for dropdown
    facility_sql = "SELECT Facility_ID FROM FACILITY"
    facilities = execute_query(facility_sql, fetchall=True)
    return render(request, 'employee/add_employee.html',
        {'facilities': facilities})

def edit_employee(request):

    if request.method == 'POST':
        # Retrieve form data
        employee_id = request.POST.get('employee_id')
        ssn = request.POST.get('ssn')
        first_name = request.POST.get('first_name')
        middle_name = request.POST.get('middle_name', '')
        last_name = request.POST.get('last_name')
```

```
street = request.POST.get('street')
city = request.POST.get('city')
state = request.POST.get('state')
zip_code = request.POST.get('zip')
salary = request.POST.get('salary')
date_hired = request.POST.get('date_hired')
job_class = request.POST.get('job_class')

# Construct kwargs based on job class
kwargs = {}
if job_class == 'Doctor':
    kwargs['speciality'] = request.POST.get('speciality')
    kwargs['board_certification_date'] =
       request.POST.get('board_certification_date')
elif job_class == 'Nurse':
    kwargs['certification'] = request.POST.get('certification')
elif job_class == 'HCP':
    kwargs['practice_area'] = request.POST.get('practice_area')
elif job_class == 'Admin':
    kwargs['job_title'] = request.POST.get('job_title')

result = update_employee_details(employee_id, job_class,
   **kwargs)

if result != "Error":
    employee_sql = """
    UPDATE EMPLOYEE
    SET SSN = %s, FirstName = %s, MiddleName = %s, LastName =
       %s, Street = %s, City = %s, State = %s, Zip = %s,
       Salary = %s, Date_Hired = %s, Job_Class = %s
    WHERE EmployeeID = %s
    """
    employee_params = (ssn, first_name, middle_name,
       last_name, street, city, state, zip_code, salary,
       date_hired, job_class, employee_id)
    result = execute_query(employee_sql,
       params=employee_params)

if result != "Error":
    messages.success(request, 'Employee details updated
       successfully.')
else:
    messages.error(request, 'Error updating employee details.
       Please try again.')

return redirect('edit_employee')
```

```
    sql = "SELECT * FROM EMPLOYEE"
    employees = execute_query(sql, fetchall=True)
    employee_id = request.GET.get('employeeIdDropdown')
    if employee_id:
        emp = get_employee_details(employee_id)
        facility_sql = "SELECT Facility_ID FROM FACILITY"
        facilities = execute_query(facility_sql, fetchall=True)
        return render(request, 'employee/edit_employee.html',
           {'employee_details': emp, 'employees': employees,
           'facilities': facilities})
    return render(request, 'employee/edit_employee.html',
       {'employees': employees})

def get_employee_details(employee_id):
        employee_query = """
        SELECT * FROM EMPLOYEE WHERE EmployeeID = %s
        """
        employee_data = execute_query(employee_query, [employee_id],
           fetchone=True)

        if employee_data:
            # Determine the job class of the employee
            job_class = employee_data['Job_Class']

            # Fetch additional details based on job class
            if job_class == 'Doctor':
                details_query = """
                SELECT Speciality, Board_Certification_Date FROM
                   DOCTOR WHERE EmployeeID = %s
                """
            elif job_class == 'Nurse':
                details_query = """
                SELECT Certification FROM NURSE WHERE EmployeeID = %s
                """
            elif job_class == 'HCP':
                details_query = """
                SELECT Practice_Area FROM OTHER_HCP WHERE EmployeeID =
                   %s
                """
            elif job_class == 'Admin':
                details_query = """
                SELECT Job_Title FROM ADMIN_STAFF WHERE EmployeeID = %s
                """

            additional_details = execute_query(details_query,
               [employee_id], fetchone=True)
```

```
            # Merge employee data with additional details
            employee_data.update(additional_details)

            return employee_data

def view_facility(request):
    sql = "SELECT * FROM FACILITY"
    facilities = execute_query(sql, fetchall=True)
    paginator = Paginator(facilities, 5)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)
    return render(request, 'facility/facility.html', {'facilities':
       page_obj})

def view_ob(request):
    sql = """
    SELECT f.*, ob.Office_Count
    FROM FACILITY f
    JOIN OFFICE_BUILDING ob ON f.Facility_ID = ob.Facility_ID
    """
    facilities = execute_query(sql, fetchall=True)
    paginator = Paginator(facilities, 5)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)
    return render(request, 'facility/office_building.html',
       {'facilities': page_obj})

def view_ops(request):
    sql = """
    SELECT f.*, ops.Room_Count, ops.Procedure_Code, ops.Description
    FROM FACILITY f
    JOIN OUTPATIENT_SURGERY ops ON f.Facility_ID = ops.Facility_ID
    """
    facilities = execute_query(sql, fetchall=True)
    paginator = Paginator(facilities, 5)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)
    return render(request, 'facility/ops.html', {'facilities':
       page_obj})

def edit_facility(request):
    if request.method == 'POST':
        try:
            # Retrieve form data
            facility_id = request.POST.get('facility_id')
            street = request.POST.get('street')
            city = request.POST.get('city')
```

```
state = request.POST.get('state')
zip_code = request.POST.get('zip')
facility_type = request.POST.get('facility_type')
max_size = request.POST.get('size')
facility_type = facility_type.replace("_", " ")

# Check if facility type has changed
current_facility_type =
   get_current_facility_type(facility_id)
if current_facility_type != facility_type:
    # Delete old entry from the specific table based on
       the current facility type
    delete_old_facility_details(facility_id,
       current_facility_type)

    # Insert new entry into the updated specific table
       based on the new facility type
    if facility_type == 'Office':
        office_count = request.POST.get('office_count')
        result = insert_office_details(facility_id,
           office_count)
    elif facility_type == 'OP Surgery':
        room_count = request.POST.get('room_count')
        procedure_code = request.POST.get('procedure_code')
        description = request.POST.get('description')
        result = insert_ops_details(facility_id,
           room_count, procedure_code, description)
else:
    # Update existing entry in the specific table based on
       the facility type
    if facility_type == 'Office':
        office_count = request.POST.get('office_count')
        result = update_office_details(facility_id,
           office_count)
    elif facility_type == 'OP Surgery':
        room_count = request.POST.get('room_count')
        procedure_code = request.POST.get('procedure_code')
        description = request.POST.get('description')
        result = update_ops_details(facility_id,
           room_count, procedure_code, description)

if result != "Error":
    # Update FACILITY table
    facility_sql = """
    UPDATE FACILITY
    SET Street = %s, City = %s, State = %s, Zip = %s,
       Facility_Type = %s, MaxSize = %s
```

```
                WHERE Facility_ID = %s
                """
                facility_params = (street, city, state, zip_code,
                   facility_type, max_size, facility_id)
                result = execute_query(facility_sql,
                   params=facility_params)

            if result != "Error":
                messages.success(request, 'Facility details updated
                   successfully.')
            else:
                messages.error(request, 'Error updating facility
                   details. Please try again.')

            return redirect('edit_facility')
        except Exception as e:
            messages.error(request, f'Error updating facility details:
               {str(e)}')
            return redirect('edit_facility')

    # Retrieve all facilities to display in a dropdown for editing
    sql = "SELECT * FROM FACILITY"
    facilities = execute_query(sql, fetchall=True)
    facility_id = request.GET.get('facilityDropdown')
    if facility_id:
        facility_details = get_facility_details(facility_id)
        return render(request, 'facility/edit_facility.html',
           {'facility_details': facility_details, 'facilities':
           facilities})

    return render(request, 'facility/edit_facility.html',
       {'facilities': facilities})

def get_facility_details(facility_id):
    # First, get the facility type
    type_sql = "SELECT Facility_Type FROM FACILITY WHERE Facility_ID =
       %s"
    facility_type = execute_query(type_sql, params=(facility_id,),
       fetchone=True)

    if facility_type['Facility_Type'] == 'OP Surgery':
        # Query details for outpatient surgery
        sql = """
        SELECT f.*, ops.Room_Count, ops.Procedure_Code, ops.Description
        FROM FACILITY f
        LEFT JOIN OUTPATIENT_SURGERY ops ON f.Facility_ID =
           ops.Facility_ID
```

```
            WHERE f.Facility_ID = %s
            """
        else:
            # Query details for office building
            sql = """
            SELECT f.*, ob.Office_Count
            FROM FACILITY f
            LEFT JOIN OFFICE_BUILDING ob ON f.Facility_ID = ob.Facility_ID
            WHERE f.Facility_ID = %s
            """

        facility_details = execute_query(sql, params=(facility_id,),
           fetchone=True)
        return facility_details

    def create_facility(request):
        if request.method == 'POST':
            street = request.POST.get('street')
            city = request.POST.get('city')
            state = request.POST.get('state')
            zip_code = request.POST.get('zip')
            size = request.POST.get('size')
            facility_type = request.POST.get('facility_type')
            facility_type = facility_type.replace("_", " ")
            sql_insert_facility = """
            INSERT INTO FACILITY (Street, City, State, Zip, MaxSize,
               Facility_Type)
            VALUES (%s, %s, %s, %s, %s,  %s)
            """
            facility_id = execute_query(sql_insert_facility, (street,
               city, state, zip_code, size, facility_type), insert_new =
               True)
            result = "Success"
            if facility_type == 'Office':
                office_count = request.POST.get('office_count')
                result = insert_office_details(facility_id, office_count)
            elif facility_type == 'OP Surgery':
                room_count = request.POST.get('room_count')
                procedure_code = request.POST.get('procedure_code')
                description = request.POST.get('description')
                result = insert_ops_details(facility_id, room_count,
                   procedure_code, description)

            if result != "Error":
                messages.success(request, 'Facility Created Successfully.')
            else:
                messages.error(request, 'Error while creating Facility.
```

```
                Please try again.')
            # Redirect to a new URL after POST
            return redirect('create_facility')
        return render(request, 'facility/add_facility.html')

def view_insurance(request):
    sql = "SELECT * FROM INSURANCE_COMPANY"
    insurance_companies = execute_query(sql, fetchall=True)
    paginator = Paginator(insurance_companies, 5)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)
    return render(request, 'insurance/view_insurance.html',
       {'insurance_companies': page_obj})

def add_insurance(request):
    if request.method == 'POST':
        name = request.POST.get('name')
        street = request.POST.get('street')
        city = request.POST.get('city')
        state = request.POST.get('state')
        zip_code = request.POST.get('zip')
        sql_add_insurance = """
        INSERT INTO INSURANCE_COMPANY (Name, Street, City, State, Zip)
        VALUES (%s, %s, %s, %s, %s)
        """
        result = execute_query(sql_add_insurance, (name, street, city,
           state, zip_code))
        if result != "Error":
            messages.success(request, 'Insurance Company Registed
               Successfully.')
        else:
            messages.error(request, 'Error while creating Insurance
               Company. Please try again.')
        return redirect('add_insurance')
    return render(request, 'insurance/add_insurance.html')

def edit_insurance(request):
    if request.method == 'POST':
        insurance_id = request.POST.get('insurance_id')
        name = request.POST.get('name')
        street = request.POST.get('street')
        city = request.POST.get('city')
        state = request.POST.get('state')
        zip_code = request.POST.get('zip')
        sql_edit_insurance = """
        UPDATE INSURANCE_COMPANY
        SET Name = %s, Street = %s, City = %s, State = %s, Zip = %s
```

```
            WHERE InsuranceComp_ID = %s
            """
            param = (name, street, city, state, zip_code, insurance_id)
            result = execute_query(sql_edit_insurance, params=param)
            if result != "Error":
                messages.success(request, 'Insurance Company Details
                   Updated Successfully.')
            else:
                messages.error(request, 'Error while updating Insurance
                   Company Details. Please try again.')
            return redirect('edit_insurance')
        sql = "SELECT * FROM INSURANCE_COMPANY"
        insurance_companies = execute_query(sql, fetchall=True)
        #print("VIEWs.py FETCHED? ",len(insurance_companies))
        insurance_id = request.GET.get('insuranceDropdown')
        #print("VIEWs.py INSURANCE ID::",insurance_id)
        if insurance_id:
            insurance_details = get_insurance_details(insurance_id)
            return render(request, 'insurance/edit_insurance.html',
               {'insurance_details': insurance_details,
               'insurance_companies': insurance_companies})
        return render(request, 'insurance/edit_insurance.html',
           {'insurance_companies': insurance_companies})

    def get_insurance_details(insurance_id):
        #print("VIEWS.py COMPANY :: ",insurance_id)
        sql = """
        SELECT * FROM INSURANCE_COMPANY WHERE InsuranceComp_ID = %s
        """
        #print("SQL is ",sql)
        insurance_details = execute_query(sql, (insurance_id),
           fetchone=True)
        #print('Insurance Details ',insurance_details)
        return insurance_details

    def view_patient(request):
        sql = "SELECT * FROM PATIENT"
        patients = execute_query(sql, fetchall=True)
        paginator = Paginator(patients, 5)
        page_number = request.GET.get('page')
        page_obj = paginator.get_page(page_number)
        return render(request, 'patient/patient.html', {'patients':
           page_obj})

    def add_patient(request):
        doctors_sql = """
                        SELECT d.EmployeeID, e.FirstName, e.LastName
```

```
                    FROM DOCTOR d
                    JOIN EMPLOYEE e ON d.EmployeeID = e.EmployeeID
                """
    doctors = execute_query(doctors_sql, fetchall=True)

    insurances_sql = "SELECT InsuranceComp_ID, Name FROM
       INSURANCE_COMPANY"
    insurances = execute_query(insurances_sql, fetchall=True)

    if request.method == 'POST':
        first_name = request.POST['first_name']
        middle_name = request.POST.get('middle_name', '')
        last_name = request.POST['last_name']
        street = request.POST['street']
        city = request.POST['city']
        state = request.POST['state']
        zip_code = request.POST['zip']
        first_visit_date = request.POST['first_visit_date']
        doctor_id = request.POST['doctor_id']
        incomp_id = request.POST['incomp_id']

        insert_patient_sql = """
        INSERT INTO PATIENT (FirstName, MiddleName, LastName, Street,
           City, State, Zip, First_Visit_Date, Doctor_ID, InComp_ID)
        VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
        """
        patient_params = (first_name, middle_name, last_name, street,
           city, state, zip_code, first_visit_date, doctor_id,
           incomp_id)
        result = execute_query(insert_patient_sql, patient_params)
        if result != "Error":
            messages.success(request, 'Patient Registed Successfully.')
        else:
            messages.error(request, 'Error while creating Patient.
               Please try again.')
        return redirect('add_patient')
    return render(request, 'patient/add_patient.html', {'doctors':
       doctors, 'insurances': insurances})

def edit_patient(request):
    if request.method == 'POST':
        patient_id = request.POST.get('patient_id')
        first_name = request.POST.get('first_name')
        middle_name = request.POST.get('middle_name', '')
        last_name = request.POST.get('last_name')
        street = request.POST.get('street')
        city = request.POST.get('city')
```

```
            state = request.POST.get('state')
            zip_code = request.POST.get('zip')
            first_visit_date = request.POST.get('first_visit_date')
            doctor_id = request.POST.get('doctorIdDropdown')
            incomp_id = request.POST.get('insuranceDropdown')
            sql = """
            UPDATE PATIENT
            SET FirstName = %s, MiddleName = %s, LastName = %s, Street =
               %s, City = %s, State = %s, Zip = %s, First_Visit_Date = %s,
               Doctor_ID = %s, InComp_ID = %s
            WHERE Patient_ID = %s
            """
            params = (first_name, middle_name, last_name, street, city,
               state, zip_code, first_visit_date, doctor_id, incomp_id,
               patient_id)
            result = execute_query(sql, params)
            if result != "Error":
                messages.success(request, 'Patient Details Updated
                   Successfully.')
            else:
                messages.error(request, 'Error while updating Patient
                   Details. Please try again.')
            return redirect('edit_patient')
        patients_sql = "SELECT Patient_ID FROM PATIENT"
        patients = execute_query(patients_sql, fetchall=True)
        patient_id = request.GET.get('patientIdDropdown')
        if patient_id:
            patient_details = get_patient_details(patient_id)
            doctors_sql = """
                        SELECT d.EmployeeID, e.FirstName, e.LastName
                        FROM DOCTOR d
                        JOIN EMPLOYEE e ON d.EmployeeID = e.EmployeeID
                    """
            doctors = execute_query(doctors_sql, fetchall=True)
            insurances_sql = "SELECT InsuranceComp_ID, Name FROM
               INSURANCE_COMPANY"
            insurances = execute_query(insurances_sql, fetchall=True)
            return render(request, 'patient/edit_patient.html',
               {'patients': patients, 'patient_details': patient_details,
               'doctors': doctors, 'insurances': insurances})
        return render(request, 'patient/edit_patient.html', {'patients':
           patients})

    def get_patient_details(patient_id):
        sql = """
        SELECT * FROM PATIENT WHERE Patient_ID = %s
        """
```

```
    patient_details = execute_query(sql, (patient_id), fetchone=True)
    return patient_details


def view_appointment(request):

    sql = """
        SELECT
        P.Patient_ID,
        CONCAT(P.FirstName, ' ', P.LastName) AS Patient_Name,
        CONCAT(E.FirstName, ' ', E.LastName) AS Doctor_Name,
        MA.Fac_ID AS Facility_ID,
        MA.Reason AS Reason,
        MA.Date_Time AS Appointment_Date_Time,
        CASE
            WHEN ID.Cost IS NULL THEN 'None'
            ELSE ID.Cost
        END AS Appointment_Cost,
        CASE
            WHEN ID.Cost IS NULL THEN 'Pending'
            ELSE 'Completed'
        END AS Status
    FROM
        MAKES_APPOINTMENT MA
    INNER JOIN
        DOCTOR D ON MA.Doc_ID = D.EmployeeID
    INNER JOIN
        EMPLOYEE E ON D.EmployeeID = E.EmployeeID
    INNER JOIN
        PATIENT P ON MA.Pat_ID = P.Patient_ID
    LEFT JOIN
        INVOICE_DETAIL ID ON MA.InD_ID = ID.InvDetailID
    ORDER BY
        MA.Date_Time DESC;
        """
    apps = execute_query(sql, fetchall=True)
    paginator = Paginator(apps, 5)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)
    return render(request, 'patient/appointments.html', {'patients':
       page_obj})

def make_appointment(request):
    if request.method == 'POST':
        patient_id = request.POST['patient_id']
        doctor_id = request.POST['doctor_id']
        appointment_date = request.POST['appointment_date']
```

```
            appointment_time = request.POST['appointment_time']
            facility_id = request.POST['facility_id']
            reason = request.POST['reason']
            datetime = f"{appointment_date} {appointment_time}:00"
            invD_ID = None
            insert_appointment_sql = """
            INSERT INTO MAKES_APPOINTMENT (Pat_ID, Doc_ID, Date_Time,
               Fac_ID, InD_ID, Reason)
            VALUES (%s, %s, %s, %s, %s, %s)
            """
            appointment_params = (patient_id, doctor_id, datetime,
               facility_id, invD_ID, reason)
            result = execute_query(insert_appointment_sql,
               appointment_params, insert_new=True)
            if result != "Error":
                messages.success(request, 'Appointment Created
                   Successfully.')
            else:
                messages.error(request, 'Error while creating appointment
                   due to conflicts. Please try again.')
            return redirect('make_appointment')
        patients_sql = "SELECT Patient_ID FROM PATIENT"
        patients = execute_query(patients_sql, fetchall=True)
        doctors_sql = "SELECT EmployeeID FROM DOCTOR"
        doctors = execute_query(doctors_sql, fetchall=True)
        facilities_sql = "SELECT Facility_ID FROM FACILITY"
        facilities = execute_query(facilities_sql, fetchall=True)
        return render(request, 'patient/make_appointment.html',
           {'patients': patients, 'doctors': doctors, 'facilities':
           facilities})

    def update_appointment(request):
        if request.method == 'POST':
                cost = request.POST['cost']
                patient_id = request.POST['patient_id']
                doctor_id = request.POST['doctor_id']
                facility_id = request.POST['facility_id']
                reason = request.POST['reason']
                appointment_date = request.POST['appointment_date']
                appointment_time = request.POST['appointment_time']
                datetime = f"{appointment_date} {appointment_time}:00"

                #old Values
                opatient_id = request.POST['opatient_id']
                odoctor_id = request.POST['odoctor_id']
                ofacility_id = request.POST['ofacility_id']
                oappointment_date = request.POST['oappointment_date']
```

```
oappointment_time = request.POST['oappointment_time']
odatetime = f"{oappointment_date} {oappointment_time}:00"

ind_id = None

if cost:
    insuranceComp_sql = """
    SELECT InComp_ID FROM PATIENT WHERE Patient_ID = %s
    """
    incomp_id = execute_query(insuranceComp_sql,
       [patient_id], fetchone=True)
    incomp_id = incomp_id['InComp_ID']

    getInvoiceID = """
    SELECT Inv_ID FROM INVOICE
    WHERE InComp_ID = %s and InvDate = %s
    """
    invoice_id = execute_query(getInvoiceID, (incomp_id,
       appointment_date), fetchone=True)
    invoiceId = invoice_id['Inv_ID'] if invoice_id else
       None
    if invoiceId is None:
        insert_invoice_sql = """
        INSERT INTO INVOICE (InvDate, InComp_ID)
        VALUES (%s, %s)
        """
        latest_invoice_id =
           execute_query(insert_invoice_sql,
           (appointment_date, incomp_id), insert_new=True)
        invoiceId = latest_invoice_id

    insert_invoice_details_sql = """
    INSERT INTO INVOICE_DETAIL (Inv_ID, Cost)
    VALUES (%s, %s)
    """
    ind_id = execute_query(insert_invoice_details_sql,
       (invoiceId, cost), insert_new=True)

update_appointment_sql = """
UPDATE MAKES_APPOINTMENT
SET InD_ID = %s, Reason = %s, Pat_ID = %s, Doc_ID = %s,
   Date_Time = %s, Fac_ID = %s
WHERE Pat_ID = %s AND Doc_ID = %s AND Date_Time = %s AND
   Fac_ID = %s
"""
appointment_params = (ind_id, reason, patient_id,
   doctor_id, datetime, facility_id, opatient_id,
```

```
            odoctor_id, odatetime, ofacility_id)
        result = execute_query(update_appointment_sql,
            appointment_params)
        if result != "Error":
            messages.success(request, 'Appointment Details Updated
                Successfully.')
        else:
            messages.error(request, 'Error while updating
                appointment details due to conflict. Please try
                again.')
        return redirect('update_appointment')

patients_sql = "SELECT Patient_ID FROM PATIENT"
patients = execute_query(patients_sql, fetchall=True)
doctors_sql = "SELECT EmployeeID FROM DOCTOR"
doctors = execute_query(doctors_sql, fetchall=True)
facilities_sql = "SELECT Facility_ID FROM FACILITY"
facilities = execute_query(facilities_sql, fetchall=True)

patient_id = request.GET.get('patient_id')
doctor_id = request.GET.get('doctor_id')
facility_id = request.GET.get('facility_id')
appointment_date = request.GET.get('appointment_date')
appointment_time = request.GET.get('appointment_time')
appointment_datetime = f"{appointment_date} {appointment_time}:00"
if patient_id and doctor_id and facility_id and appointment_date
    and appointment_time:
    appointment_sql = """
        SELECT * FROM MAKES_APPOINTMENT
        WHERE Pat_ID = %s AND Doc_ID = %s AND Date_Time = %s AND
            Fac_ID = %s
    """
    appointment = execute_query(appointment_sql, (patient_id,
        doctor_id, appointment_datetime, facility_id),
        fetchone=True)

    if (appointment):
        Date_time = appointment['Date_Time']
        appointment['appointment_date_str'] =
            Date_time.strftime('%Y-%m-%d')
        appointment['appointment_time_str'] =
            Date_time.strftime('%H:%M')
        if(appointment['InD_ID'] is None):
            return render(request,
                'patient/update_appointment.html', {'patients':
                patients, 'doctors': doctors, 'facilities':
                facilities, 'appointment': appointment})
```

```
            else:
                return render(request,
                   'patient/update_appointment.html', {'patients':
                   patients, 'doctors': doctors, 'facilities':
                   facilities, 'appointment': appointment, 'hascost':
                   True})
    return render(request, 'patient/update_appointment.html',
       {'patients': patients, 'doctors': doctors, 'facilities':
       facilities})

def daily_inv(request):
    sql = """
            SELECT p.`Patient_ID`, p.`FirstName`, i.`InvDate`,
               ic.`Name`, SUM(id.`Cost`) AS 'TotalCost'
            FROM `PATIENT` p
            JOIN `MAKES_APPOINTMENT` ma ON p.`Patient_ID` = ma.`Pat_ID`
            JOIN `INVOICE_DETAIL` id ON ma.`InD_ID` = id.`InvDetailID`
            JOIN `INVOICE` i ON id.`Inv_ID` = i.`Inv_ID`
            JOIN `INSURANCE_COMPANY` ic ON p.`InComp_ID` =
               ic.`InsuranceComp_ID`
            GROUP BY i.`InvDate`, ic.`Name`, p.`Patient_ID`
            ORDER BY i.`InvDate` DESC
        """
    costs = execute_query(sql, fetchall=True)
    paginator = Paginator(costs, 5)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)
    return render(request, 'patient/daily_inv.html', {'costs':
       page_obj})

def revenue_report_by_facility(request):
    selected_date = request.GET.get('selected_date')
    if (request.method == 'GET' and selected_date):
        sql = """
        SELECT
            SUM(INVOICE_DETAIL.Cost) AS Total_Revenue
        FROM
            MAKES_APPOINTMENT
            INNER JOIN INVOICE_DETAIL ON MAKES_APPOINTMENT.InD_ID =
               INVOICE_DETAIL.InvDetailID
        WHERE
            DATE(MAKES_APPOINTMENT.Date_Time) = %s
        """
        total_revenue = execute_query(sql, params=(selected_date,),
           fetchone=True)['Total_Revenue']
        sql = """
        SELECT
```

```
            FACILITY.Facility_ID,
            FACILITY.City,
            FACILITY.State,
            SUM(INVOICE_DETAIL.Cost) AS Total_Revenue
        FROM
            MAKES_APPOINTMENT
            INNER JOIN INVOICE_DETAIL ON MAKES_APPOINTMENT.InD_ID =
               INVOICE_DETAIL.InvDetailID
            INNER JOIN FACILITY ON MAKES_APPOINTMENT.Fac_ID =
               FACILITY.Facility_ID
        WHERE
            DATE(MAKES_APPOINTMENT.Date_Time) = %s
        GROUP BY
            FACILITY.Facility_ID,
            FACILITY.City,
            FACILITY.State
        """
        revenue_by_facility = execute_query(sql,
           params=(selected_date,), fetchall=True)
        paginator = Paginator(revenue_by_facility, 5)
        page_number = request.GET.get('page')
        page_obj = paginator.get_page(page_number)
        return render(request, 'reports/reports1.html',
           {'revenue_by_facility': page_obj, 'selected_date':
           selected_date, 'total_revenue': total_revenue})

    return render(request, 'reports/reports1.html')

def appointments_by_date_and_physician(request):
    # Fetch physician IDs for dropdown
    physician_sql = "SELECT EmployeeID FROM Employee WHERE Job_Class
       IN ('Doctor')"
    physicians = execute_query(physician_sql, fetchall=True)
    selected_date = request.GET.get('selected_date')
    if request.method == 'GET' and selected_date:
        physician_id = request.GET.get('physician_id')
        sql0 = """
        SELECT
            EMPLOYEE.EmployeeID,
            CONCAT(EMPLOYEE.FirstName, ' ', EMPLOYEE.LastName) AS
               Employee_Name,
            CASE
                WHEN EMPLOYEE.Job_Class = 'Doctor' THEN
                   DOCTOR.Speciality
                WHEN EMPLOYEE.Job_Class = 'HCP' THEN
                   OTHER_HCP.Practice_Area
            END AS Job_Specific_Details,
```

```
    CASE
        WHEN EMPLOYEE.Job_Class = 'Doctor' THEN
           DOCTOR.Board_Certification_Date
        WHEN EMPLOYEE.Job_Class = 'HCP' THEN NULL
    END AS Certification_Date
FROM
    EMPLOYEE
    LEFT JOIN DOCTOR ON EMPLOYEE.EmployeeID = DOCTOR.EmployeeID
    LEFT JOIN OTHER_HCP ON EMPLOYEE.EmployeeID =
       OTHER_HCP.EmployeeID
WHERE
EMPLOYEE.EmployeeID = %s"""
doc_details = execute_query(sql0, params=(physician_id),
   fetchone=True)

sql = """
    SELECT
    MAKES_APPOINTMENT.*,
    PATIENT.FirstName AS Patient_FirstName,
    PATIENT.LastName AS Patient_LastName,
    CONCAT(FACILITY.Street, ', ', FACILITY.City, ', ',
       FACILITY.State, ' ', FACILITY.Zip) AS Facility_Address,
    CASE
        WHEN MAKES_APPOINTMENT.InD_ID IS NOT NULL THEN
           'Completed'
        ELSE 'Pending'
    END AS Status
FROM
    MAKES_APPOINTMENT
    INNER JOIN PATIENT ON MAKES_APPOINTMENT.Pat_ID =
       PATIENT.Patient_ID
    INNER JOIN FACILITY ON MAKES_APPOINTMENT.Fac_ID =
       FACILITY.Facility_ID
WHERE
    DATE(MAKES_APPOINTMENT.Date_Time) = %s
    AND MAKES_APPOINTMENT.Doc_ID = %s

"""
appointments = execute_query(sql, params=(selected_date,
   physician_id), fetchall=True)
paginator = Paginator(appointments, 5)
page_number = request.GET.get('page')
page_obj = paginator.get_page(page_number)

return render(request, 'reports/reports2.html', {'physician':
   doc_details, 'appointments': page_obj, 'employees':
   physicians, 'selected_date': selected_date})
```

```
    return render(request, 'reports/reports2.html', {'employees':
        physicians})

def appointments_by_time_period_and_facility(request):
    # Fetch facility IDs for dropdown
    facility_sql = "SELECT Facility_ID FROM FACILITY"
    facilities = execute_query(facility_sql, fetchall=True)
    selected_start_date = request.GET.get('selected_start_date')
    selected_end_date = request.GET.get('selected_end_date')
    selected_facility_id = request.GET.get('selected_facility_id')
    if request.method == 'GET' and selected_start_date and
        selected_end_date and selected_facility_id:
        sql = """
        SELECT DATE(ma.Date_Time) as Date, TIME(ma.Date_Time) as Time,
            d.EmployeeID as Doctor_ID, e.FirstName as Doctor,
           p.Patient_ID as Patient_ID, p.FirstName as Patient,
           ma.Reason as Description
        FROM MAKES_APPOINTMENT ma
        JOIN DOCTOR d ON ma.Doc_ID = d.EmployeeID
        JOIN EMPLOYEE e ON d.EmployeeID = e.EmployeeID
        JOIN PATIENT p ON ma.Pat_ID = p.Patient_ID
        JOIN FACILITY f ON ma.Fac_ID = f.Facility_ID
        WHERE (ma.Date_Time BETWEEN %s AND %s) AND f.Facility_ID = %s
        """
        appointments = execute_query(sql, params=(selected_start_date,
            selected_end_date, selected_facility_id), fetchall=True)
        paginator = Paginator(appointments, 5)
        page_number = request.GET.get('page')
        page_obj = paginator.get_page(page_number)
        return render(request, 'reports/reports3.html',
            {'appointments': page_obj, 'facilities': facilities,
            'selected_start_date':selected_start_date,
            'selected_end_date':selected_end_date,
            'selected_facility_id':selected_facility_id})

    return render(request, 'reports/reports3.html', {'facilities':
        facilities})

def best_revenue_days_for_month(request):
    selected_month = request.GET.get('selected_month')
    selected_year = request.GET.get('selected_year')
    print('selected_month', selected_month)
    print('selected_year', selected_year)
    if request.method == 'GET' and selected_month and selected_year:
        print('selected_month', selected_month)
        print('selected_year', selected_year)
```

```
        sql = """
        SELECT SUM(inv.`Total_Cost`) as Total_Revenue, inv.`InvDate`
           as InvDate
        FROM `INVOICE` inv
        WHERE YEAR(inv.`InvDate`) = %s and MONTH(inv.`InvDate`) = %s
        GROUP BY inv.`InvDate`
        ORDER BY Total_Revenue DESC
        LIMIT 5
        """
        best_days = execute_query(sql,
           params=(selected_year,selected_month,), fetchall=True)
        return render(request, 'reports/reports4.html', {'best_days':
           best_days,
           'selected_month':selected_month,'selected_year':selected_year})
    return render(request, 'reports/reports4.html')

def average_daily_revenue_by_insurance_company(request):
    selected_start_date = request.GET.get('selected_start_date')
    selected_end_date = request.GET.get('selected_end_date')
    if request.method == 'GET' and selected_start_date and
       selected_end_date:
        print('selected_start_date', selected_start_date)
        print('selected_end_date', selected_end_date)
        number_of_days =
           (datetime.datetime.strptime(selected_end_date, '%Y-%m-%d')
           - datetime.datetime.strptime(selected_start_date,
           '%Y-%m-%d')).days + 1

        sql = """
            SELECT ic.Name as Name, SUM(inv.Total_Cost) / %s as
               Average_Cost
            FROM INVOICE inv
            JOIN INSURANCE_COMPANY ic ON inv.InComp_ID =
               ic.InsuranceComp_ID
            WHERE inv.InvDate BETWEEN %s AND %s
            GROUP BY inv.InComp_ID;
        """
        average_daily_revenue = execute_query(sql,
           params=(number_of_days, selected_start_date,
           selected_end_date), fetchall=True)
        paginator = Paginator(average_daily_revenue, 5)
        page_number = request.GET.get('page')
        page_obj = paginator.get_page(page_number)

        return render(request, 'reports/reports5.html',
           {'average_revenue': page_obj, 'selected_start_date':
           selected_start_date, 'selected_end_date':
```

```
            selected_end_date})

    return render(request, 'reports/reports5.html')
```

5.5 manageEmployee.py

```
        from .db_utils import execute_query


def get_current_job_class(employee_id):
    # Query the database to retrieve the current job class of the
       employee
    sql = "SELECT Job_Class FROM EMPLOYEE WHERE EmployeeID = %s"
    result = execute_query(sql, params=(employee_id,), fetchone=True)
    if result:
        return result['Job_Class']
    return None

def delete_old_entry(employee_id, current_job_class):
    # Delete the old entry from the respective table based on the
       current job class
    if current_job_class == 'Doctor':
        sql = "DELETE FROM DOCTOR WHERE EmployeeID = %s"
    elif current_job_class == 'Nurse':
        sql = "DELETE FROM NURSE WHERE EmployeeID = %s"
    elif current_job_class == 'HCP':
        sql = "DELETE FROM OTHER_HCP WHERE EmployeeID = %s"
    elif current_job_class == 'Admin':
        sql = "DELETE FROM ADMIN_STAFF WHERE EmployeeID = %s"
    execute_query(sql, params=(employee_id,))

def insert_doctor_details(employee_id, speciality,
   board_certification_date):
    # Insert new entry into the Doctor table
    sql = """
    INSERT INTO DOCTOR (EmployeeID, Speciality,
       Board_Certification_Date)
    VALUES (%s, %s, %s)
    """
    params = (employee_id, speciality, board_certification_date)
    return execute_query(sql, params=params)

def update_doctor_details(employee_id, speciality,
   board_certification_date):
    # Update existing entry in the Doctor table
    sql = """
    UPDATE DOCTOR
```

```
    SET Speciality = %s, Board_Certification_Date = %s
    WHERE EmployeeID = %s
    """
    params = (speciality, board_certification_date, employee_id)
    return execute_query(sql, params=params)

def insert_nurse_details(employee_id, certification):
    # Insert new entry into the Nurse table
    sql = """
    INSERT INTO NURSE (EmployeeID, Certification)
    VALUES (%s, %s)
    """
    params = (employee_id, certification)
    return execute_query(sql, params=params)

def update_nurse_details(employee_id, certification):
    # Update existing entry in the Nurse table
    sql = """
    UPDATE NURSE
    SET Certification = %s
    WHERE EmployeeID = %s
    """
    params = (certification, employee_id)
    return execute_query(sql, params=params)

def insert_other_hcp_details(employee_id, practice_area):
    # Insert new entry into the Other HCP table
    sql = """
    INSERT INTO OTHER_HCP (EmployeeID, Practice_Area)
    VALUES (%s, %s)
    """
    params = (employee_id, practice_area)
    return execute_query(sql, params=params)

def update_other_hcp_details(employee_id, practice_area):
    # Update existing entry in the Other HCP table
    sql = """
    UPDATE OTHER_HCP
    SET Practice_Area = %s
    WHERE EmployeeID = %s
    """
    params = (practice_area, employee_id)
    return execute_query(sql, params=params)

def insert_admin_staff_details(employee_id, job_title):
    # Insert new entry into the Admin Staff table
    sql = """
```

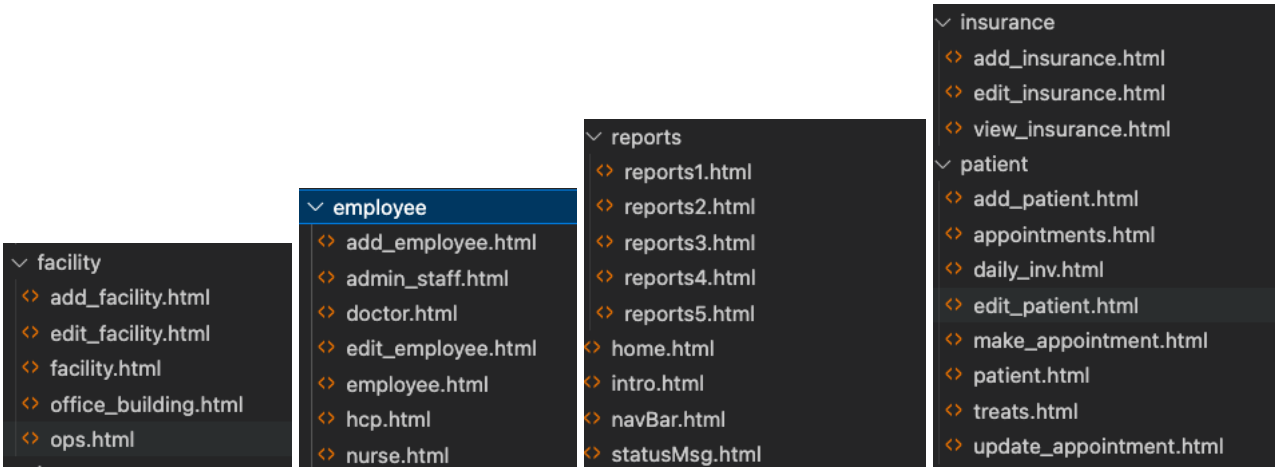
```
    INSERT INTO ADMIN_STAFF (EmployeeID, Job_Title)
    VALUES (%s, %s)
    """
    params = (employee_id, job_title)
    return execute_query(sql, params=params)

def update_admin_staff_details(employee_id, job_title):
    # Update existing entry in the Admin Staff table
    sql = """
    UPDATE ADMIN_STAFF
    SET Job_Title = %s
    WHERE EmployeeID = %s
    """
    params = (job_title, employee_id)
    return execute_query(sql, params=params)


def update_employee_details(employee_id, job_class, **kwargs):
    result = None

    current_job_class = get_current_job_class(employee_id)
    if current_job_class != job_class:
        print(current_job_class, job_class)
        delete_old_entry(employee_id, current_job_class)

        # Insert new entry into the updated class table
        if job_class == 'Doctor':
            result = insert_doctor_details(employee_id, **kwargs)
        elif job_class == 'Nurse':
            result = insert_nurse_details(employee_id, **kwargs)
        elif job_class == 'HCP':
            result = insert_other_hcp_details(employee_id, **kwargs)
        elif job_class == 'Admin':
            result = insert_admin_staff_details(employee_id, **kwargs)
    else:
        # Update existing entry in the same class table
        if job_class == 'Doctor':
            result = update_doctor_details(employee_id, **kwargs)
        elif job_class == 'Nurse':
            result = update_nurse_details(employee_id, **kwargs)
        elif job_class == 'HCP':
            result = update_other_hcp_details(employee_id, **kwargs)
        elif job_class == 'Admin':
            result = update_admin_staff_details(employee_id, **kwargs)

    return result
```

5.6 manageFacility.py

```
from .db_utils import execute_query
def get_current_facility_type(facility_id):
    # Query the database to retrieve the current facility type of the
       facility
    sql = "SELECT Facility_Type FROM FACILITY WHERE Facility_ID = %s"
    result = execute_query(sql, params=(facility_id,), fetchone=True)
    if result:
        return result['Facility_Type']
    return None

def delete_old_facility_details(facility_id, current_facility_type):
    # Delete the old entry from the specific table based on the
       current facility type
    if current_facility_type == 'Office':
        sql = "DELETE FROM OFFICE_BUILDING WHERE Facility_ID = %s"
    elif current_facility_type == 'OP Surgery':
        sql = "DELETE FROM OUTPATIENT_SURGERY WHERE Facility_ID = %s"
    execute_query(sql, params=(facility_id,))

def insert_office_details(facility_id, office_count):
    # Insert new entry into the Office Building table
    sql = """
    INSERT INTO OFFICE_BUILDING (Facility_ID, Office_Count)
    VALUES (%s, %s)
    """
    params = (facility_id, office_count)
    return execute_query(sql, params=params)

def insert_ops_details(facility_id, room_count, procedure_code,
   description):
    # Insert new entry into the Outpatient Surgery table
    sql = """
    INSERT INTO OUTPATIENT_SURGERY (Facility_ID, Room_Count,
       Procedure_Code, Description)
    VALUES (%s, %s, %s, %s)
    """
    params = (facility_id, room_count, procedure_code, description)
    return execute_query(sql, params=params)

def update_office_details(facility_id, office_count):
    # Update existing entry in the Office Building table
    sql = """
    UPDATE OFFICE_BUILDING
    SET Office_Count = %s
```

```
    WHERE Facility_ID = %s
    """
    params = (office_count, facility_id)
    return execute_query(sql, params=params)

def update_ops_details(facility_id, room_count, procedure_code,
    description):
    # Update existing entry in the Outpatient Surgery table
    sql = """
    UPDATE OUTPATIENT_SURGERY
    SET Room_Count = %s, Procedure_Code = %s, Description = %s
    WHERE Facility_ID = %s
    """
    params = (room_count, procedure_code, description, facility_id)
    return execute_query(sql, params=params)
```

5.7 Html templates file structure for our UI by functionality as given in the question with our github repository (link).



USER GUIDE

1. The first screen is the home screen for the MHS application. The navigation bar can help you view different sections of the application. This can be accessed from every tab specified in the application. Each tab that is selected is highlighted and the sub tabs for under that section (on the left hand side of the screen) handle their specific operations.
2. Note that the screens associated with each functionality clearly indicate the fields that are required to be either entered or selected. Some common fields include State that takes only 2 characters (eg NJ for New Jersey), ZipCodes have to be 5 digit numerics, etc.
3. For each entity that is recorded under MHS, their IDs are generated by the database and will be displayed whenever necessary. This ID cannot be edited.
4. Clicking on Employees, a user can view all the employees by their categories, edit their details and add new employees too. Their corresponding details need to be entered through forms, while creating and to edit an employee, their Employee ID needs to be selected.
5. Similar to Employees and its subcategories like Doctor, Nurses, etc., we have Facilities tab that aids in their management with create, update and view operations for each category.
6. To view, edit or add insurance companies associated with MHS, use the Insurance Tab.
7. Under Patients, you may create, edit or view patients associated with Doctors at MHS and their Insurance Companies. Along with that, you can create Appointments to handle their interactions with various physicians at multiple facilities for a given date and time. You may edit these fields as long as an appointment's Status is Pending. An appointment is completed when you enter the cost associated with it. These costs are reflected in the Invoices sub tab.
8. Invoices are associated with an Insurance Company and a daily record (if needed) is maintained. If there are no appointments for a day, there are no invoices for that day.
9. The reports tab helps you view various statistics for selected fields associated with that tab. The application helps you view:
    a. Revenue Generated by a Facility for a specific date with facility subtotals and a total value too.
    b. Appointments for a given date and physician.
    c. Appointment details for a given time period and facility.
    d. Best Five Revenue days for MHS given a month and a year.
    e. Average daily revenue for each Insurance Company given a time period.

The UI screens from here onwards represent each of these functionalities in detail.

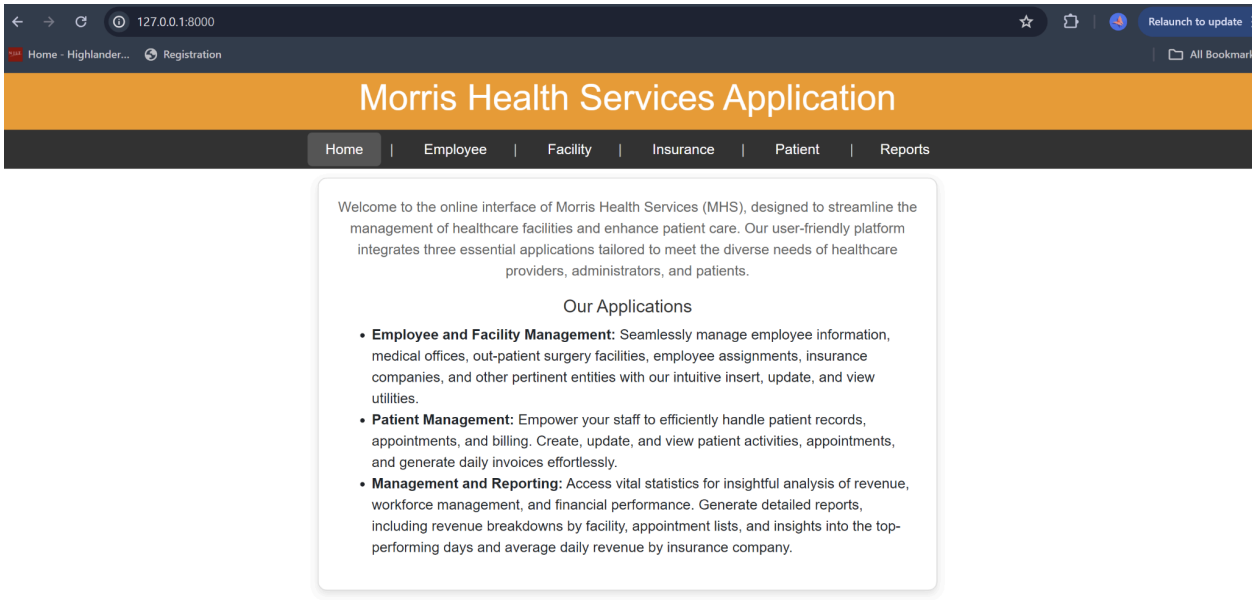


Figure 1: Home Tab of our Application

Employee Management

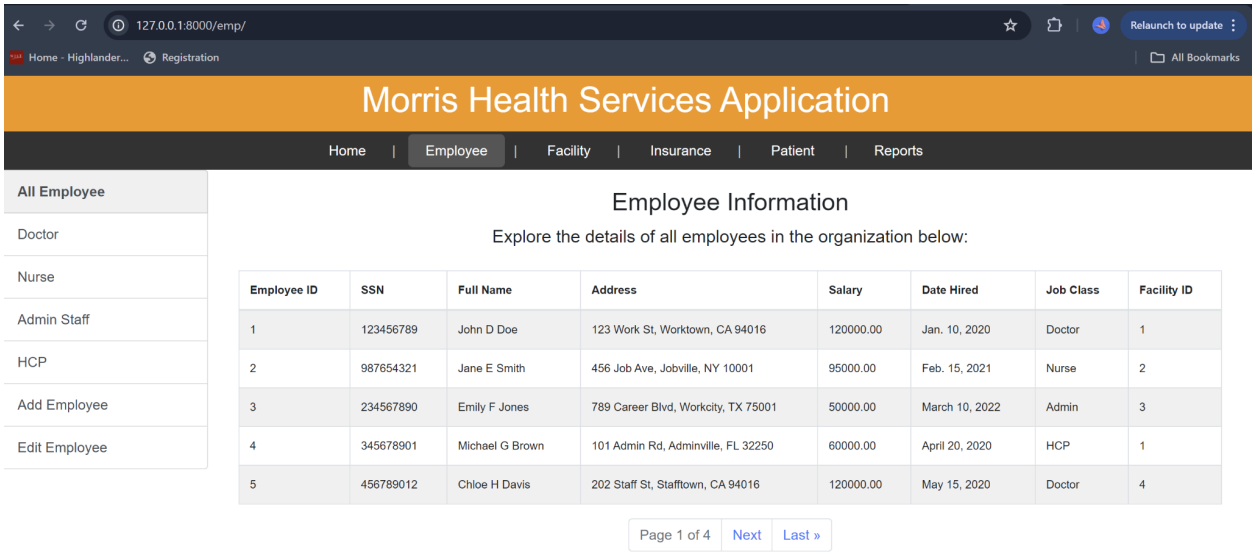


Figure 2: View All Employee Screen

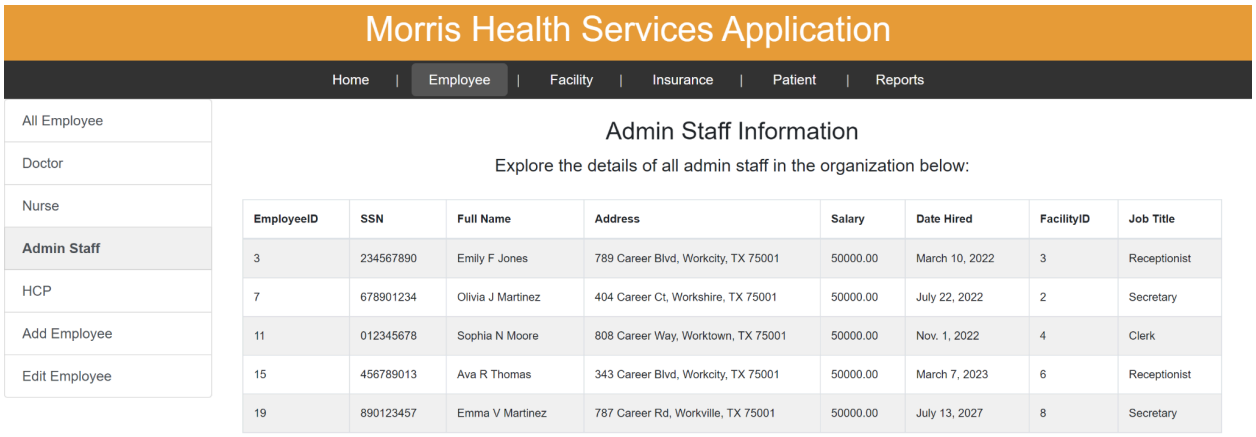


Figure 3: View All Admin Staff Screen

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- All Employee
- Doctor
- Nurse
- Admin Staff
- **HCP**
- Add Employee
- Edit Employee

Health Care Professionals Information

Explore the details of all health care professionals in the organization below:

EmployeeID	SSN	Full Name	Address	Salary	Date Hired	FacilityID	Practice Area
4	345678901	Michael G Brown	101 Admin Rd, Adminville, FL 32250	60000.00	April 20, 2020	1	Physical Therapy
8	789012345	James K Taylor	505 Admin Ave, Adminburg, FL 32250	60000.00	Aug. 25, 2020	3	Occupational Therapy
12	123456780	Jacob O Jackson	909 Admin St, Admincity, FL 32250	60000.00	Dec. 3, 2020	5	Speech Therapy
16	567890124	Noah S Miller	454 Admin Way, Adminburgh, FL 32250	60000.00	April 8, 2024	7	Physical Therapy
20	901234568	Jack W Taylor	898 Admin Ln, Adminland, FL 32250	60000.00	Aug. 14, 2028	9	Occupational Therapy

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- All Employee
- Doctor
- **Nurse**
- Admin Staff
- HCP
- Add Employee
- Edit Employee

Nurses Information

Explore the details of all nurses in the organization below:

EmployeeID	SSN	Full Name	Address	Salary	Date Hired	FacilityID	Certification
2	987654321	Jane E Smith	456 Job Ave, Jobville, NY 10001	95000.00	Feb. 15, 2021	2	RN
6	567890123	Luke I Wilson	303 Job Way, Jobburgh, NY 10001	95000.00	June 18, 2021	5	LPN
10	901234567	Ethan M Harris	707 Job Ln, Jobland, NY 10001	95000.00	Oct. 30, 2021	7	CNA
14	345678902	William Q Anderson	232 Job Rd, Jobville, NY 10001	95000.00	Feb. 5, 2022	9	RN
18	789012346	Benjamin U Wilson	676 Job Ave, Jobshire, NY 10001	95000.00	June 11, 2026	11	LPN

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- All Employee
- Doctor
- Nurse
- Admin Staff
- HCP
- **Add Employee**
- Edit Employee

Create New Employee

SSN: 123459876

First Name: Sruthi

Middle Name:

Last Name: Vellore

Street: 456 Job Ave

City: Jobville

State: NY

Zip: 10001

Salary: 95000

Date Hired: 04/29/2024

Job Class: Doctor

Facility ID: 12

Speciality: Hematology

Board Certification Date: 09/12/2022

Submit

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- All Employee
- Doctor
- Nurse
- Admin Staff
- HCP
- **Add Employee**
- Edit Employee

Create New Employee

SSN:

First Name:

Middle Name:

Last Name:

Street:

City:

State:

Zip:

Salary:

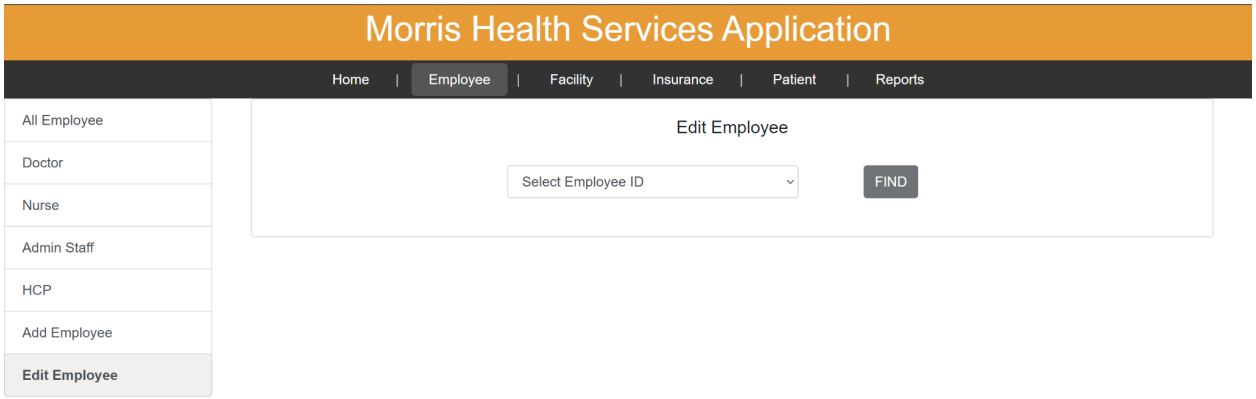
Date Hired: mm/dd/yyyy

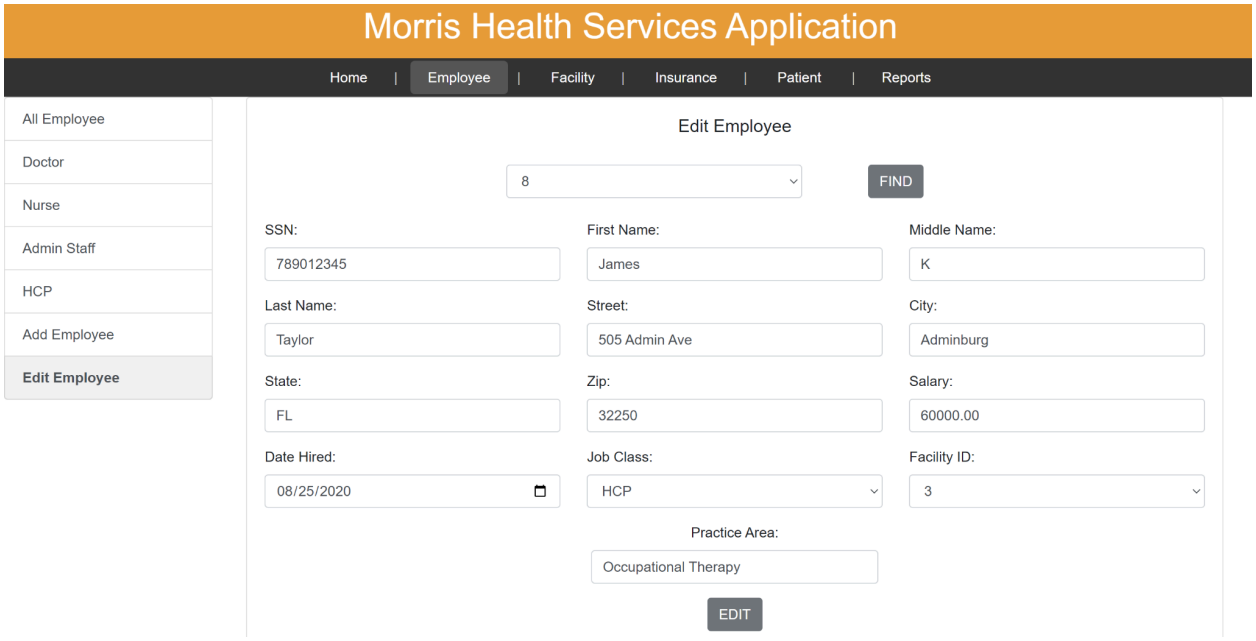
Job Class: Select

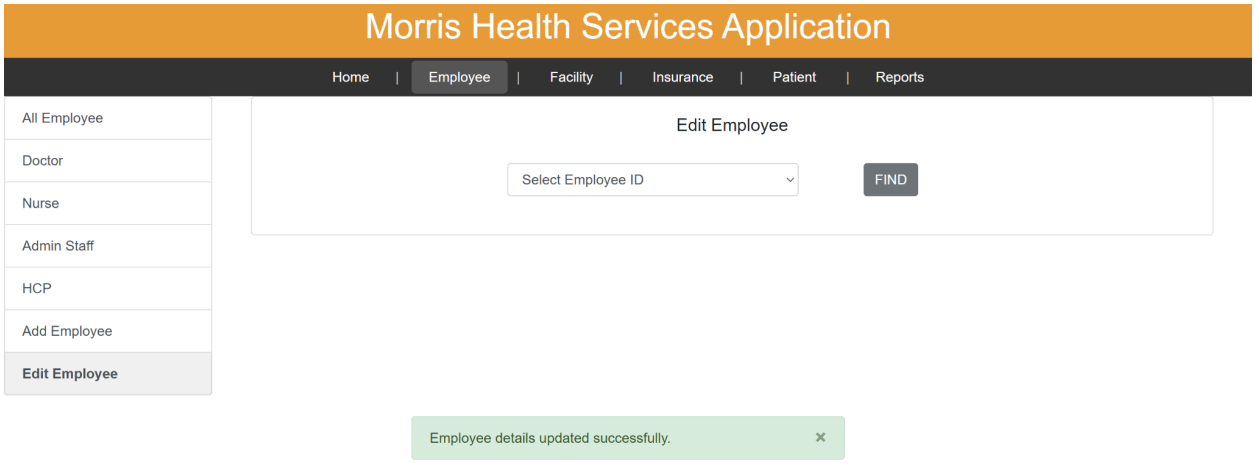
Facility ID: Select

Submit

Employee details inserted successfully. ×


Morris Health Services Application
Home | Employee | Facility | Insurance | Patient | Reports
All Employee
Doctor
Nurse
Admin Staff
HCP
Add Employee
Edit Employee
Edit Employee
Select Employee ID
FIND


Morris Health Services Application
Home | Employee | Facility | Insurance | Patient | Reports
All Employee
Doctor
Nurse
Admin Staff
HCP
Add Employee
Edit Employee
Edit Employee
8
FIND
SSN:
789012345
First Name:
James
Middle Name:
K
Last Name:
Taylor
Street:
505 Admin Ave
City:
Adminburg
State:
FL
Zip:
32250
Salary:
60000.00
Date Hired:
08/25/2020
Job Class:
HCP
Facility ID:
3
Practice Area:
Occupational Therapy
EDIT


Morris Health Services Application
Home | Employee | Facility | Insurance | Patient | Reports
All Employee
Doctor
Nurse
Admin Staff
HCP
Add Employee
Edit Employee
Edit Employee
Select Employee ID
FIND
Employee details updated successfully.

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

Facilities List
Office Building
Outpatient Surgery
Add Facility
Edit Facility

Facility Information

Explore the details of all Facilities of the organization below:

Facility ID	Address	Max Size	Facility Type
1	123 Main St, Springfield, IL 62701	150	Office
2	124 Main St, Springfield, IL 62702	100	Office
3	125 Main St, Springfield, IL 62703	200	Office
4	126 Main St, Springfield, IL 62704	120	Office
5	127 Main St, Springfield, IL 62705	130	Office

Page 1 of 4 Next Last »

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

Facilities List
Office Building
Outpatient Surgery
Add Facility
Edit Facility

Office Buildings Information

Explore the details of all Office Buildings of the organization below:

Facility ID	Address	Facility Type	Office Count
1	123 Main St, Springfield, IL 62701	Office	10
2	124 Main St, Springfield, IL 62702	Office	9
3	125 Main St, Springfield, IL 62703	Office	8
4	126 Main St, Springfield, IL 62704	Office	7
5	127 Main St, Springfield, IL 62705	Office	6

Page 1 of 3 Next Last »

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

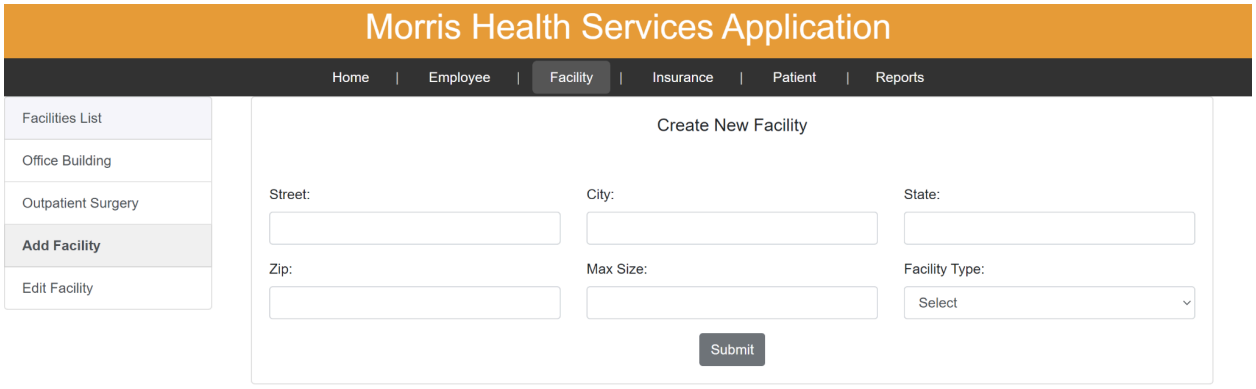
Facilities List
Office Building
Outpatient Surgery
Add Facility
Edit Facility

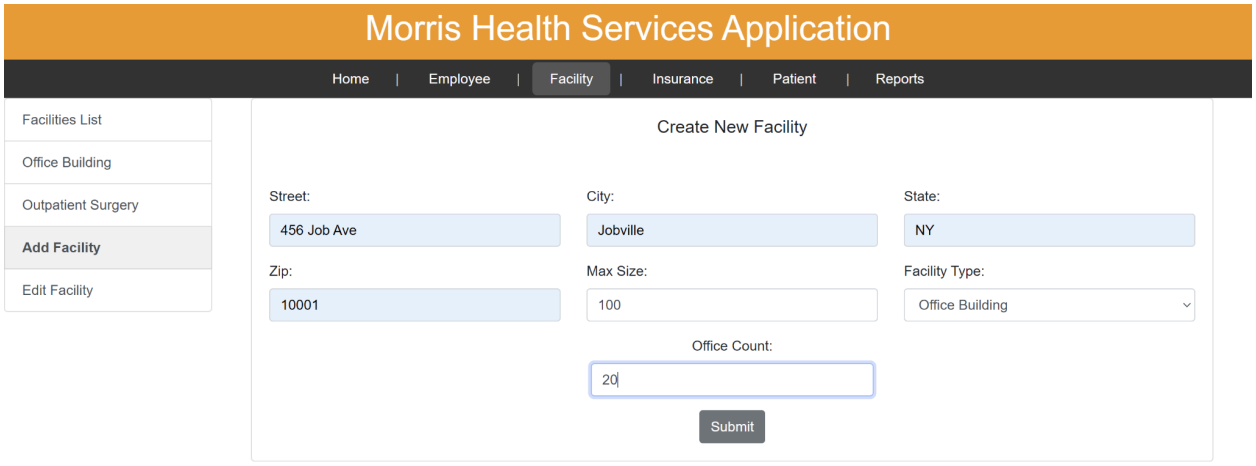
Outpatient Surgery Facility Information

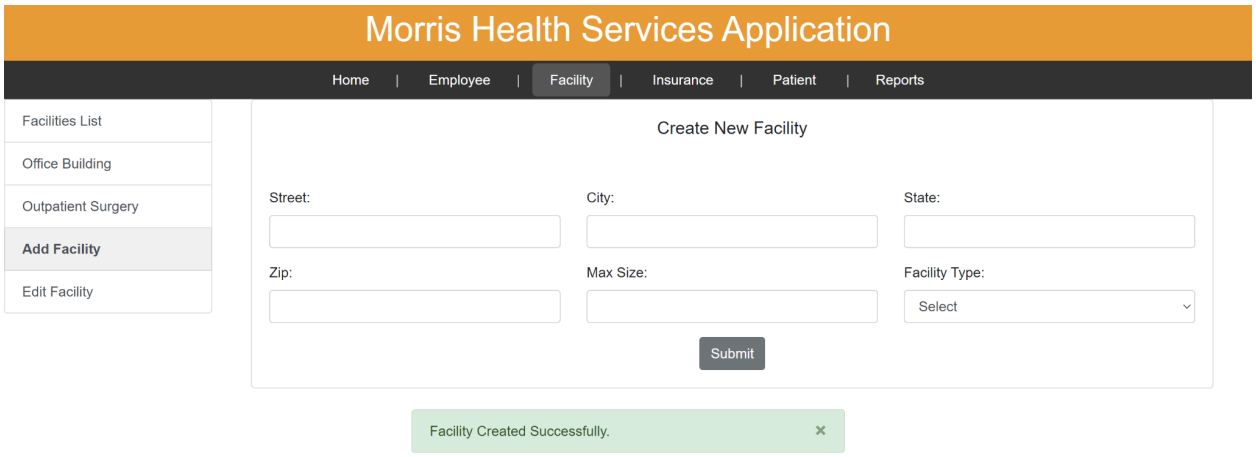
Explore the details of all Outpatient Surgery Facilities of the organization below:

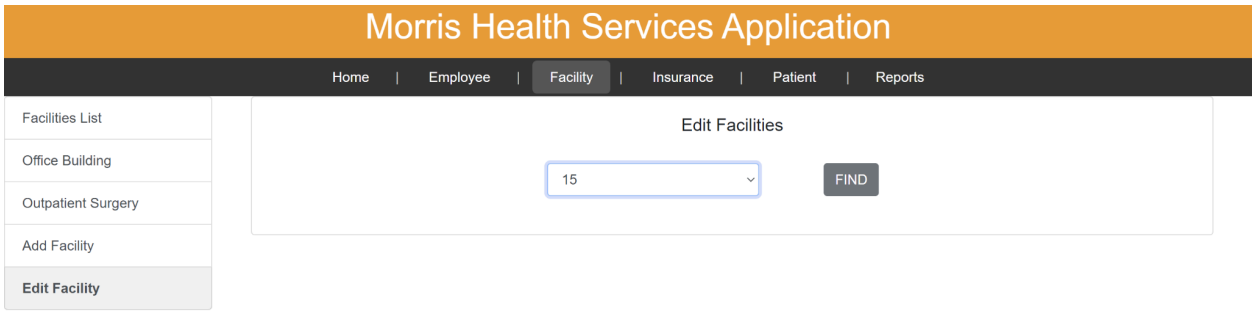
Facility ID	Address	Facility Type	Room Count	Procedure Code	Description
12	134 Pine St, Lakeview, TX 75001	OP Surgery	10	12345	Routine Appendectomy
13	135 Pine St, Lakeview, TX 75002	OP Surgery	9	23456	Cataract Surgery
14	136 Pine St, Lakeview, TX 75003	OP Surgery	8	34567	Knee Arthroscopy
15	137 Pine St, Lakeview, TX 75004	OP Surgery	7	45678	Laser Eye Surgery
16	138 Pine St, Lakeview, TX 75005	OP Surgery	6	56789	Gallbladder Removal

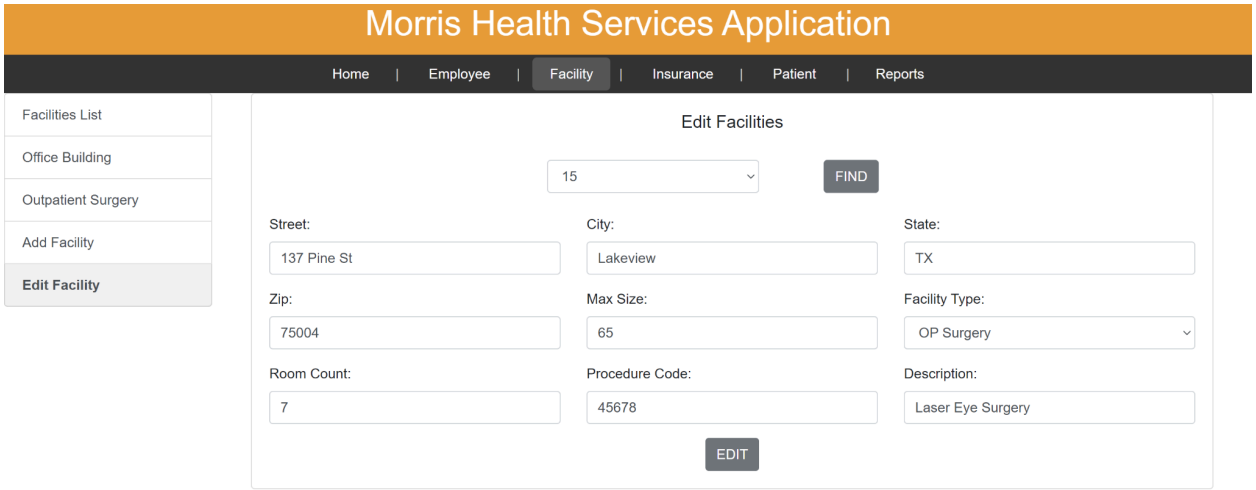
Page 1 of 2 Next Last »

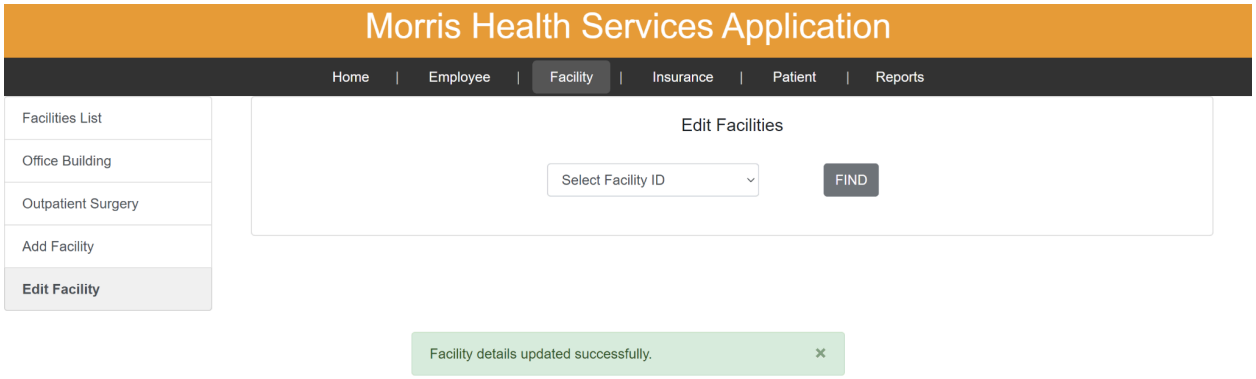

Morris Health Services Application
Home | Employee | Facility | Insurance | Patient | Reports
Facilities List
Office Building
Outpatient Surgery
Add Facility
Edit Facility
Create New Facility
Street:
City:
State:
Zip:
Max Size:
Facility Type:
Select
Submit


Morris Health Services Application
Home | Employee | Facility | Insurance | Patient | Reports
Facilities List
Office Building
Outpatient Surgery
Add Facility
Edit Facility
Create New Facility
Street:
456 Job Ave
City:
Jobville
State:
NY
Zip:
10001
Max Size:
100
Facility Type:
Office Building
Office Count:
20
Submit


Morris Health Services Application
Home | Employee | Facility | Insurance | Patient | Reports
Facilities List
Office Building
Outpatient Surgery
Add Facility
Edit Facility
Create New Facility
Street:
City:
State:
Zip:
Max Size:
Facility Type:
Select
Submit
Facility Created Successfully.


Morris Health Services Application
Home | Employee | Facility | Insurance | Patient | Reports
Facilities List
Office Building
Outpatient Surgery
Add Facility
Edit Facility
Edit Facilities
15
FIND


Morris Health Services Application
Home | Employee | Facility | Insurance | Patient | Reports
Facilities List
Office Building
Outpatient Surgery
Add Facility
Edit Facility
Edit Facilities
15
FIND
Street:
137 Pine St
City:
Lakeview
State:
TX
Zip:
75004
Max Size:
65
Facility Type:
OP Surgery
Room Count:
7
Procedure Code:
45678
Description:
Laser Eye Surgery
EDIT


Morris Health Services Application
Home | Employee | Facility | Insurance | Patient | Reports
Facilities List
Office Building
Outpatient Surgery
Add Facility
Edit Facility
Edit Facilities
Select Facility ID
FIND
Facility details updated successfully.

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- **All Insurance Companies**
- Add a Company
- Edit a Company

View Insurance Company

Explore the details of all insurance companies associated with the organization below.

ID	Name	Street	City	State	Zip
1	GoodHealth Ins	100 Health Blvd	Capital City	DC	20001
2	SecureLife	200 Wellness Dr	Oldtown	OK	73034
3	FamilyCare	300 Safe St	Trustville	CA	94016
4	BudgetShield	400 Cover Rd	Savecity	NY	10001

Page 1 of 1

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- All Insurance Companies
- **Add a Company**
- Edit a Company

Add Insurance Company

Enter the details of the insurance company below.

Name: HealthCarePVT

Street: 456 Job Ave

City: Jobville

State: NY

Zip: 10001

Submit

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- All Insurance Companies
- **Add a Company**
- Edit a Company

Add Insurance Company

Enter the details of the insurance company below.

Name:

Street:

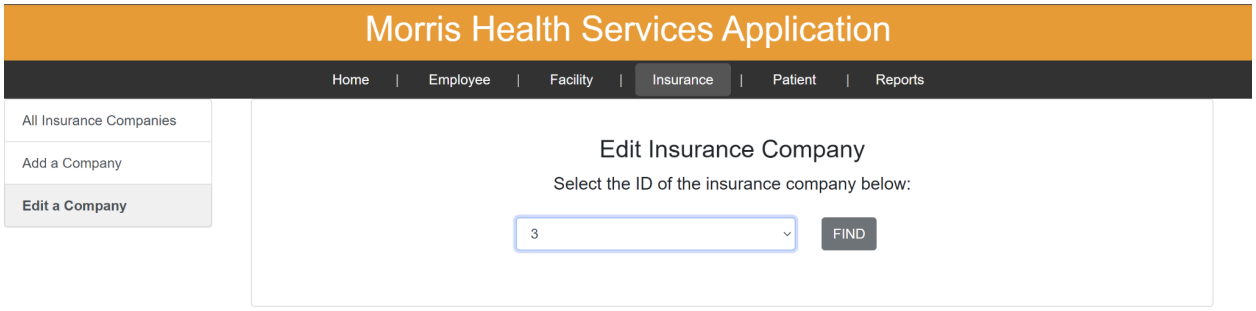
City:

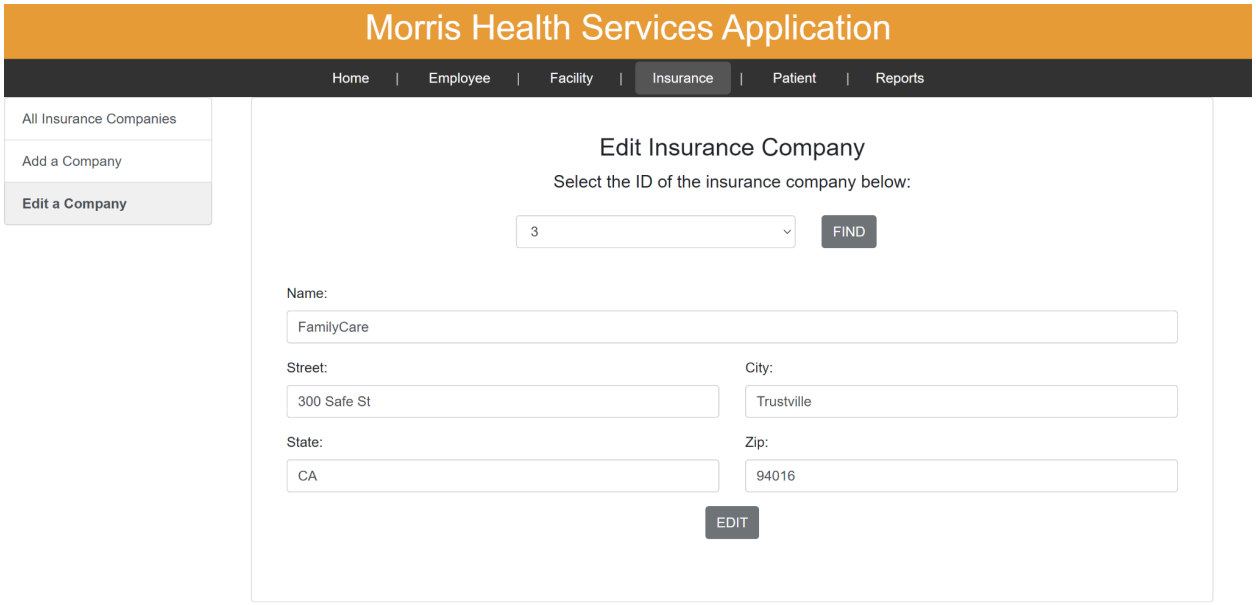
State:

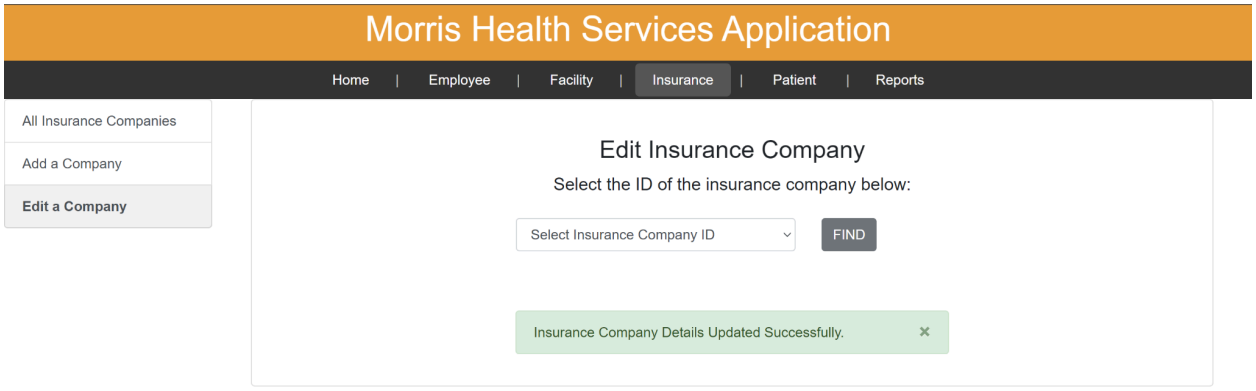
Zip:

Submit

Insurance Company Registed Successfully. ×


Morris Health Services Application
Home | Employee | Facility | Insurance | Patient | Reports
All Insurance Companies
Add a Company
Edit a Company
Edit Insurance Company
Select the ID of the insurance company below:
3
FIND


Morris Health Services Application
Home | Employee | Facility | Insurance | Patient | Reports
All Insurance Companies
Add a Company
Edit a Company
Edit Insurance Company
Select the ID of the insurance company below:
3
FIND
Name:
FamilyCare
Street:
300 Safe St
City:
Trustville
State:
CA
Zip:
94016
EDIT


Morris Health Services Application
Home | Employee | Facility | Insurance | Patient | Reports
All Insurance Companies
Add a Company
Edit a Company
Edit Insurance Company
Select the ID of the insurance company below:
Select Insurance Company ID
FIND
Insurance Company Details Updated Successfully.

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- **All Patients**
- Add Patient
- Edit Patient
- All Appointments
- Create Appointment
- Update Appointment
- Invoices

Patient Information

Explore the details of all patients in the organization below:

Patient ID	Full Name	Address	City	State	Zip	First Visit Date	Doctor Assigned	Insurance Company
1	Alice F Johnson	789 Patient Rd	Healtown	FL	32250	Jan. 10, 2023	1	1
2	Bob G Lee	101 Recovery Ln	Medcity	TX	75070	Feb. 20, 2023	5	1
3	Charlie H Clark	234 Health St	Wellville	CA	94016	March 15, 2023	9	2
4	Diana I Wright	345 Care Ave	Curetown	NY	10001	April 10, 2023	13	2
5	Evan J Morris	456 Wellness Blvd	Aidcity	VA	24502	May 5, 2023	17	3

Page 1 of 2 Next Last »

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- All Patients
- **Add Patient**
- Edit Patient
- All Appointments
- Create Appointment
- Update Appointment
- Invoices

Create New Patient

First Name: Nidhi

Middle Name:

Last Name: Vellore

Street: 456 Job Ave

City: Jobville

State: NY

Zip: 10001

First Visit Date: 04/28/2024

Doctor: 9

Insurance Company: HealthCarePVT

Submit

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- All Patients
- **Add Patient**
- Edit Patient
- All Appointments
- Create Appointment
- Update Appointment
- Invoices

Create New Patient

First Name:

Middle Name:

Last Name:

Street:

City:

State:

Zip:

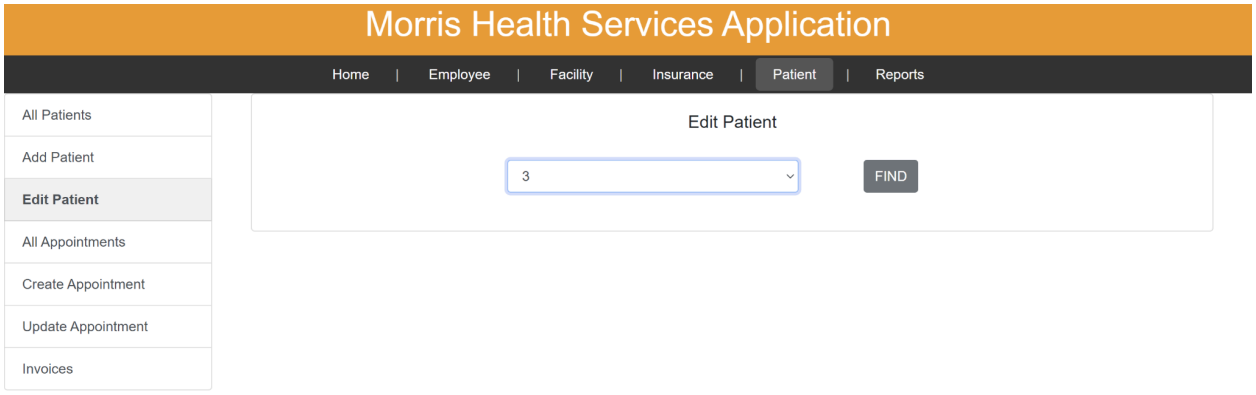
First Visit Date: mm/dd/yyyy

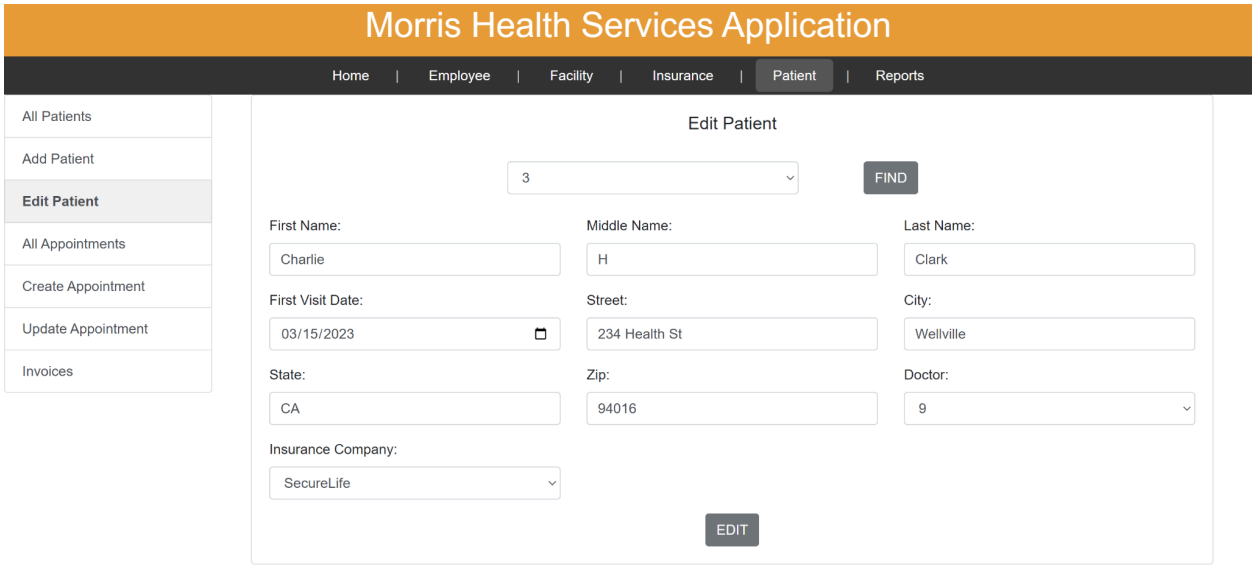
Doctor: Select

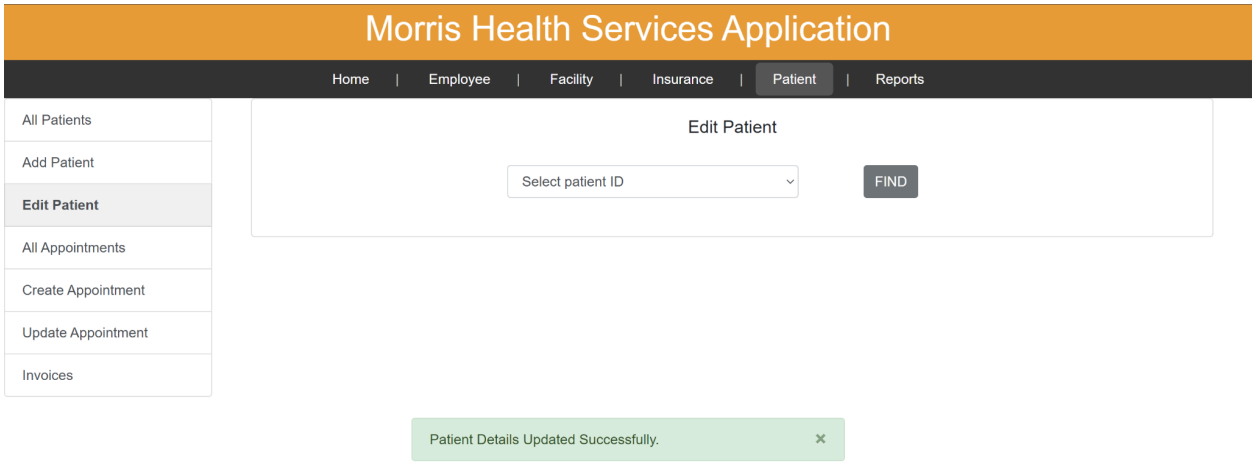
Insurance Company: Select

Submit

Patient Registed Successfully. ×


Morris Health Services Application
Home | Employee | Facility | Insurance | Patient | Reports
All Patients
Add Patient
Edit Patient
All Appointments
Create Appointment
Update Appointment
Invoices
Edit Patient
3
FIND


Morris Health Services Application
Home | Employee | Facility | Insurance | Patient | Reports
All Patients
Add Patient
Edit Patient
All Appointments
Create Appointment
Update Appointment
Invoices
Edit Patient
3
FIND
First Name:
Charlie
Middle Name:
H
Last Name:
Clark
First Visit Date:
03/15/2023
Street:
234 Health St
City:
Wellville
State:
CA
Zip:
94016
Doctor:
9
Insurance Company:
SecureLife
EDIT


Morris Health Services Application
Home | Employee | Facility | Insurance | Patient | Reports
All Patients
Add Patient
Edit Patient
All Appointments
Create Appointment
Update Appointment
Invoices
Edit Patient
Select patient ID
FIND
Patient Details Updated Successfully.

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- All Patients
- Add Patient
- Edit Patient
- **All Appointments**
- Create Appointment
- Update Appointment
- Invoices

Appointments Information

Explore the details of all Appointments scheduled:

Patient ID	Patient Name	Doctor Assigned	Facility ID	Description	Date Time	Cost	Status
3	Charlie Clark	Isabella Thomas	15	Allergies	May 3, 2024, 3:33 a.m.	453.00	Completed
3	Charlie Clark	Isabella Thomas	9	Anxiety	May 3, 2024, 3 a.m.	1000.00	Completed
10	Julia Martinez	Mia Lee	4	Asthma	May 2, 2024, 4 a.m.	1000.00	Completed
1	Alice Johnson	John Doe	1	Stomachache	May 1, 2024, 9:55 a.m.	345.00	Completed
5	Evan Morris	Chloe Davis	5	Common Cold	May 1, 2024, 8 a.m.	42.00	Completed

Page 1 of 4 Next Last »

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- All Patients
- Add Patient
- Edit Patient
- All Appointments
- **Create Appointment**
- Update Appointment
- Invoices

Create New Appointment

Patient ID: 11

Doctor ID: 21

Facility ID: 10

Date: 04/30/2024

Description: Fever

Time: 11:30 AM

Schedule

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- All Patients
- Add Patient
- Edit Patient
- All Appointments
- **Create Appointment**
- Update Appointment
- Invoices

Create New Appointment

Patient ID: Select

Doctor ID: Select

Facility ID: Select

Date: mm/dd/yyyy

Description:

Time: --:-- --

Schedule

Appointment Created Successfully. ×

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- All Patients
- Add Patient
- Edit Patient
- All Appointments
- Create Appointment
- **Update Appointment**
- Invoices

Update Appointment Details & Cost

Patient ID: 11

Doctor ID: 21

Facility ID: 10

Appointment Date: 04/30/2024

Appointment Time: 11:30 AM

Find Appointment

Edit the Appointment Details / Add Invoice Cost to Complete the Appointment.

Patient ID: 1

Doctor ID: 1

Facility ID: 10

Appointment Date: 04/30/2024

Appointment Time: 11:30 AM

Description: Fever

Cost of the Appointment:

Completed

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- All Patients
- Add Patient
- Edit Patient
- All Appointments
- Create Appointment
- **Update Appointment**
- Invoices

Update Appointment Details & Cost

Patient ID: Select

Doctor ID: Select

Facility ID: Select

Appointment Date: mm/dd/yyyy

Appointment Time: --:-- --

Find Appointment

Appointment Details Updated Successfully. ×

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- All Patients
- Add Patient
- Edit Patient
- All Appointments
- Create Appointment
- Update Appointment
- **Invoices**

View Daily Invoices for Insurance Companies

with Patient Subtotals.

Date	Insurance	Patient ID	Patient	Total Cost
May 2, 2024	SecureLife	3	Charlie	1453.00
May 2, 2024	SecureLife	10	Julia	1000.00
May 1, 2024	FamilyCare	5	Evan	42.00
May 1, 2024	GoodHealth Ins	1	Alice	876.00
April 30, 2024	GoodHealth Ins	1	Alice	1521.00

Page 1 of 3 Next Last »

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- **Facility Revenue**
- Doctor Appointments
- Facility Appointments
- Top Revenue Days
- Avg.Revenue by Ins. Co.

Facilities Revenue Report

View revenue generated by facilities for a selected day :

Total revenue generated by facilities on 2024-04-30 : $1989.00

04/30/2024 FIND

Facility ID	City	State	Total Revenue
1	Springfield	IL	$521.00
19	Lakeview	TX	$468.00
5	Springfield	IL	$1000.00

Page 1 of 1

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- Facility Revenue
- **Doctor Appointments**
- Facility Appointments
- Top Revenue Days
- Avg.Revenue by Ins. Co.

Doctor's Appointment List

1 04/30/2024 FIND

Doctor ID: 1 **Doctor Name:** John Doe **Speciality / Practice Area:** Cardiology

Facility ID	Patient ID	Patient Name	Description	Date Time	Facility Address	Status
1	1	Alice, Johnson	Back Pain	April 30, 2024, 7:45 a.m.	123 Main St, Springfield, IL 62701	Completed
10	1	Alice, Johnson	Fever	April 30, 2024, 11:30 a.m.	132 Main St, Springfield, IL 62710	Pending

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- Facility Revenue
- Doctor Appointments
- **Facility Appointments**
- Top Revenue Days
- Avg.Revenue by Ins. Co.

Facility Appointments List

Select Facility: 7 Start Date: 04/01/2024 End Date: 04/30/2024

FIND

Date	Time	Doctor ID	Doctor Name	Patient ID	Patient Name	Description
April 29, 2024	15:11:00	5	Chloe	4	Diana	Sprained Ankle

Page 1 of 1

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- Facility Revenue
- Doctor Appointments
- Facility Appointments
- **Top Revenue Days**
- Avg.Revenue by Ins. Co.

Best Five Revenue Days

Select a month below to get top 5 revenue days.

April | 2024 | Submit

Date	Revenue
April 30, 2024	2748.00
April 29, 2024	2106.00
April 28, 2024	1087.00

Morris Health Services Application

Home | Employee | Facility | Insurance | Patient | Reports

- Facility Revenue
- Doctor Appointments
- Facility Appointments
- Top Revenue Days
- **Avg.Revenue by Ins. Co.**

Average Daily Revenue from Insurance Company

View average daily revenue for a selected period :

Start Date: 04/25/2024 End Date: 04/30/2024

FIND

Name	Average Cost
GoodHealth Ins	$656.666667
SecureLife	$276.500000
FamilyCare	$57.000000

Page 1 of 1